Big Data y Minería de Datos

Unidad 2. Reglas en la minería de datos

U2.2. Sistemas basados en reglas

UNIVERSIDAD DE
MURCIA

Unidad 2. Reglas en la minería de datos
Estructura

UNIVERSIDAD DE MURCIA

- Fundamentos de los sistemas basados en reglas (SBR)
- Arquitectura de un SBR
- Inferencia
- Resolución de conflictos
- Ventajas y desventajas de sistemas expertos

Big Data y Minería de Datos
Contenidos

UNIVERSIDAD DE MURCIA

Introducción a la minería de datos

Técnicas de inteligencia artificial

Reglas
Fundamentos lógicos
Sistemas basados en reglas
Reglas de asociación

Aprendizaje computacional
Fundamentos y evaluación
Regresión y clasificación
No supervisado

Procesado del lenguaje natural
Fundamentos
Representación y tópicos
Sentimientos y NER

Introducción al big data

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

FUNDAMENTOS DE LOS SISTEMAS DE REGLAS



Unidad 2. Reglas en la minería de datos
¿Qué entendemos por conocimiento experto?

UNIVERSIDAD DE MURCIA

- Los sistemas expertos aparecen en el 1970 cuando se acepta que las máquinas puedan resolver problemas intelectuales
- Conocimiento como la comprensión teórica o práctica de un tema o dominio (puede hacer cosas que otros no)
  - Suelen ser alcances limitados, no amplios
- El proceso de pensamiento humano es demasiado complejo
  - Pero la mayoría de expertos lo pueden expresar en reglas para resolver problemas
  - Por ej, imagina que conoces un alien, ¿cómo le explicas como cruzar la calle?
- Nos vamos a basar en estructuras IF-THEN (SI-ENTONCES) que llamaremos reglas de producción o solo reglas

UNIVERSIDAD DE MURCIA

- Vamos a utilizar reglas para representar el conocimiento experto – **Sistema Basado en Reglas (SBR)**
- Las reglas se componen de dos partes
  - La parte IF (SI), llamada antecedente (premisa o condición)
  - La parte THEN (ENTONCES), llamada consecuente (conclusión o acción)
- Una regla puede tener múltiples antecedentes unidos por la conjunción AND (^), disyunción OR (ᵛ) o una combinación

IF	<antecedent 1>
AND	<antecedent 2>
	.
	.
	.
AND	<antecedent *n*>
THEN	<consequent>

IF	<antecedent 1>
OR	<antecedent 2>
	.
	.
	.
OR	<antecedent *n*>
THEN	<consequent>

IF	<antecedent>
THEN	<consequent 1>
	<consequent 2>
	.
	.
	.
	<consequent *m*>

- El antecedente de una regla tiene dos partes:
  - Un objeto (objeto lingüístico) y el valor de dicho objeto
  - Ambos están enlazados por un operador (por ejemplo, *is*, *are*, *is not*, *are not*) para asignar un valor simbólico
  - También se usan operadores matemáticos

```
IF      'age of the customer' < 18
AND     'cash withdrawal' > 1000
THEN    'signature of the parent' is required
```

  - Podemos decir, que el antecedente combina un objeto y un valor, conectados por un operador (lógico, matemático, …)

```
IF      'taxable income' > 16283
THEN    'Medicare levy' = 'taxable income' * 1.5 / 100
```

UNIVERSIDAD DE MURCIA

- Las reglas pueden representar relaciones, directivas, recomendaciones, estrategias o heurísticas (Durkin, 1994)
  - ¿Qué se os ocurre en algún dominio que seáis expertos?

Relation

IF the 'fuel tank' is empty
THEN the car is dead

Directive

IF the car is dead
AND the 'fuel tank' is empty
THEN the action is 'refuel the car'

Heuristic

IF the spill is liquid
AND the 'spill pH' < 6
AND the 'spill smell' is vinegar
THEN the 'spill material' is 'acetic acid'

Recommendation

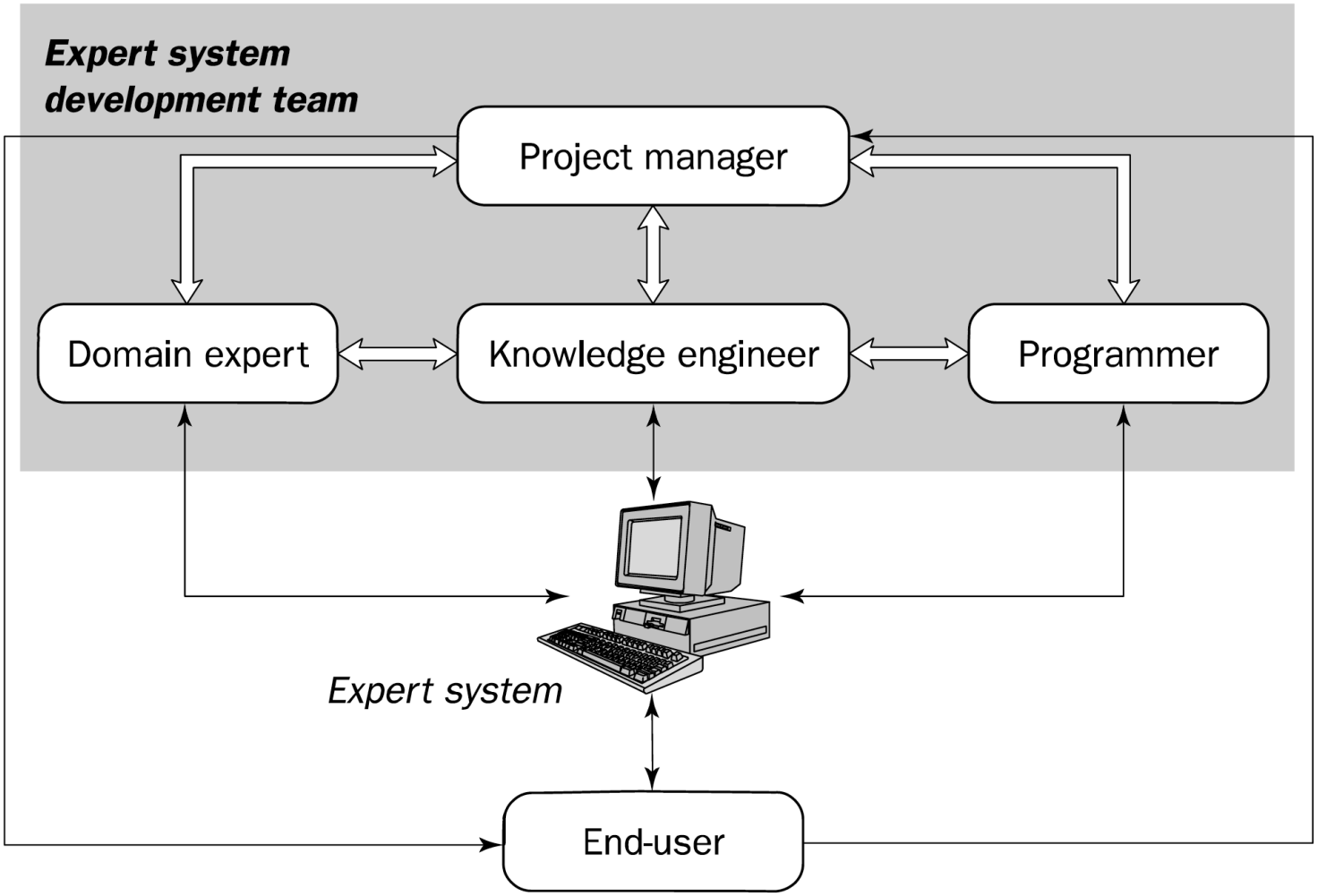
IF the season is autumn
AND the sky is cloudy
AND the forecast is drizzle
THEN the advice is 'take an umbrella'

Strategy

IF the car is dead
THEN the action is 'check the fuel tank';
step1 is complete

IF step1 is complete
AND the 'fuel tank' is full
THEN the action is 'check the battery';
step2 is complete

- La carcasa de un sistema experto, se puede considerar así incluso sin conocimiento
  - Sólo hay que añadirle las reglas para resolver el problema



- Desde un punto de vista tecnológico, las reglas son:
  - Sencillas: Representan el conocimiento de una forma sencilla
  - Atómicas: No dependen de otras reglas
- Separa el conocimiento del dominio de como se utiliza
- Se aplica la inferencia lógica: obtener una conclusión a partir de evidencias (hechos) disponibles
  - Mediante razonamiento deductivo, adaptando LP y LPO

Modus Ponens:

1. P →Q
2. P true
3. Entonces Q es true

Modus Tollens:

1. P → Q
2. Q false
3. Entonces P es false

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

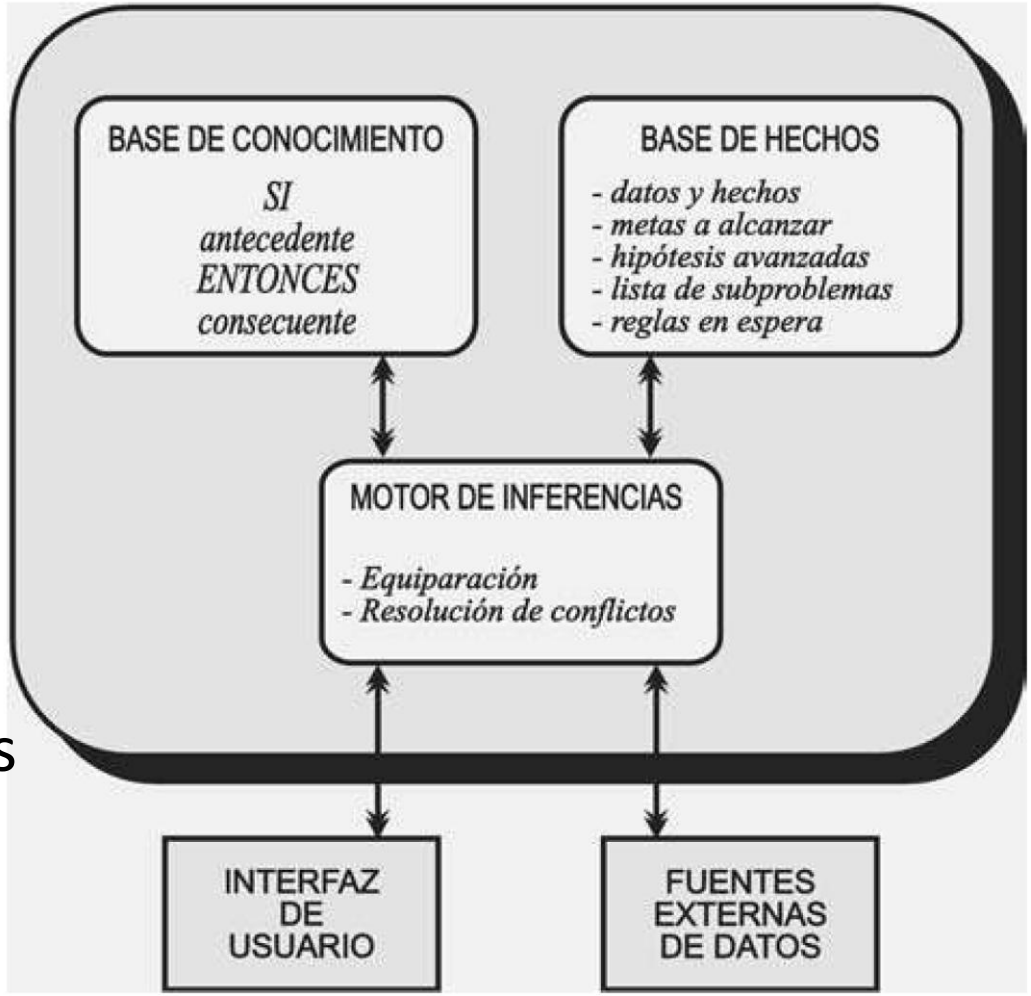
ARQUITECTURA DE UN SISTEMA BASADO EN REGLAS



Unidad 2. Reglas en la minería de datos
¿Qué es un SBR y sus componentes?

UNIVERSIDAD DE MURCIA

- Es un sistema que tiene:
  - Una base de conocimiento con reglas (BC)
  - Algún mecanismo de inferencia (MI) que selecciona reglas y las ejecuta para obtener nuevos hechos
  - Una base de hechos (BH) que acumula un conjunto de hechos establecidos (estado actual)
    - Pueden ser iniciales o generados por inferencia

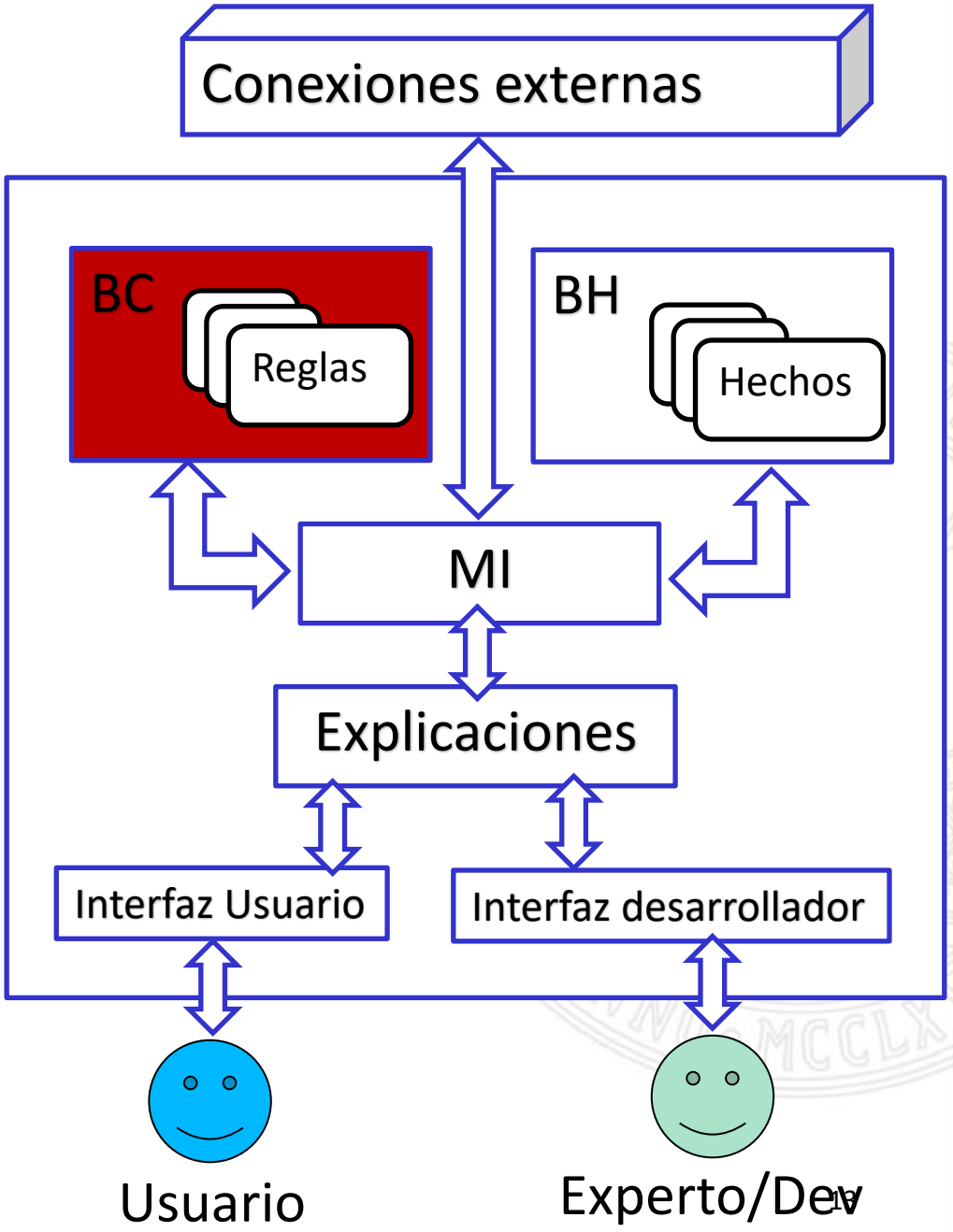


- Para que sea útil, necesitará una interfaz de usuario y de entrada/salida (a fuentes de datos externas)

Unidad 2. Reglas en la minería de datos
Base de conocimiento (BC)

UNIVERSIDAD DE MURCIA

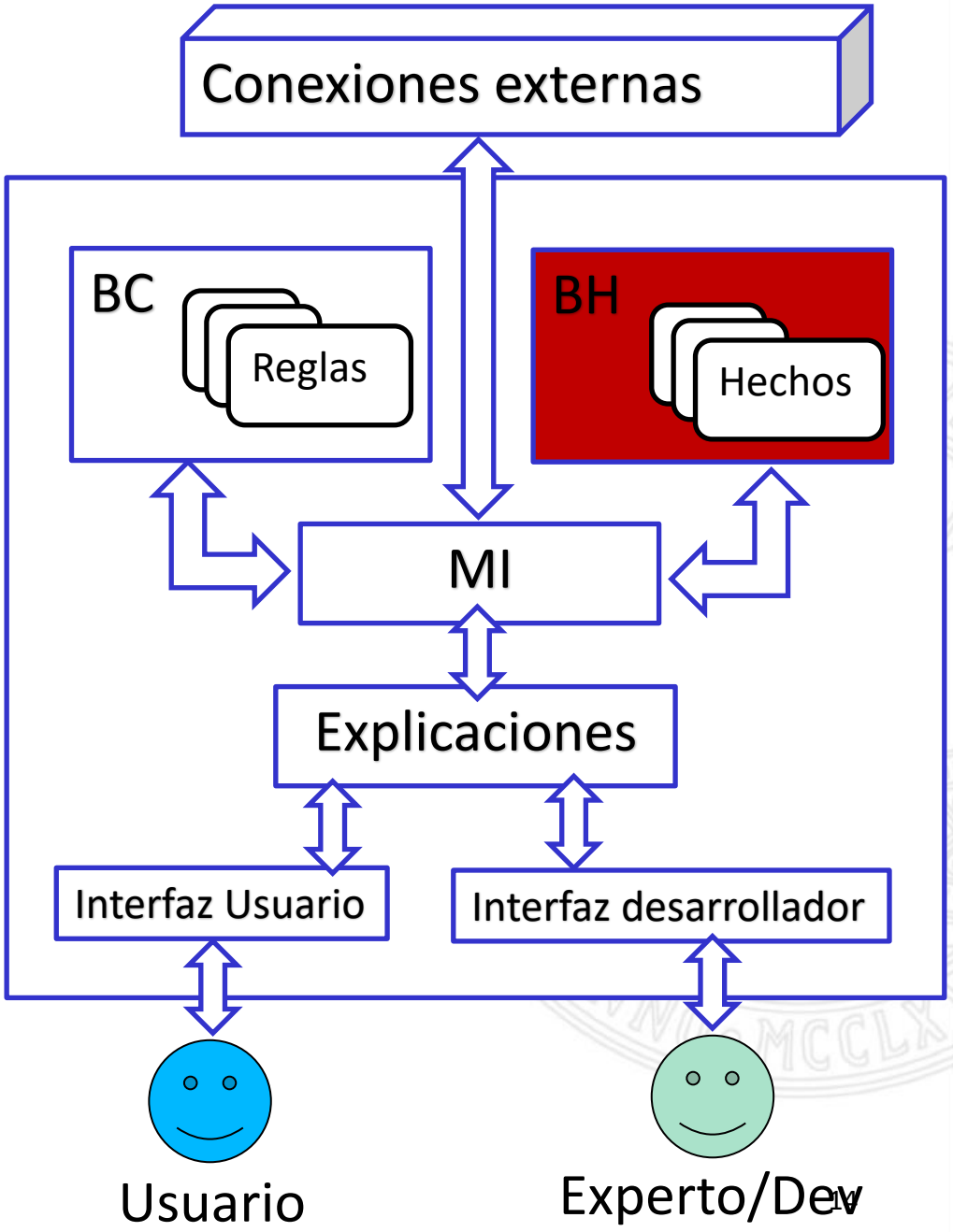
- Knowledge Base (KB, almacena reglas)
- Estructura regla: LHS → RHS
- Antecedente o Left-hand Side (LHS)
  - Condiciones que deben cumplirse mirando la BH
- Consecuente o Right-hand side (RHS)
  - Acciones a ejecutar (modifican la BH)
- Regla activada: Las condiciones de la regla se confirman



Unidad 2. Reglas en la minería de datos
Base de hechos (BH)

UNIVERSIDAD DE MURCIA

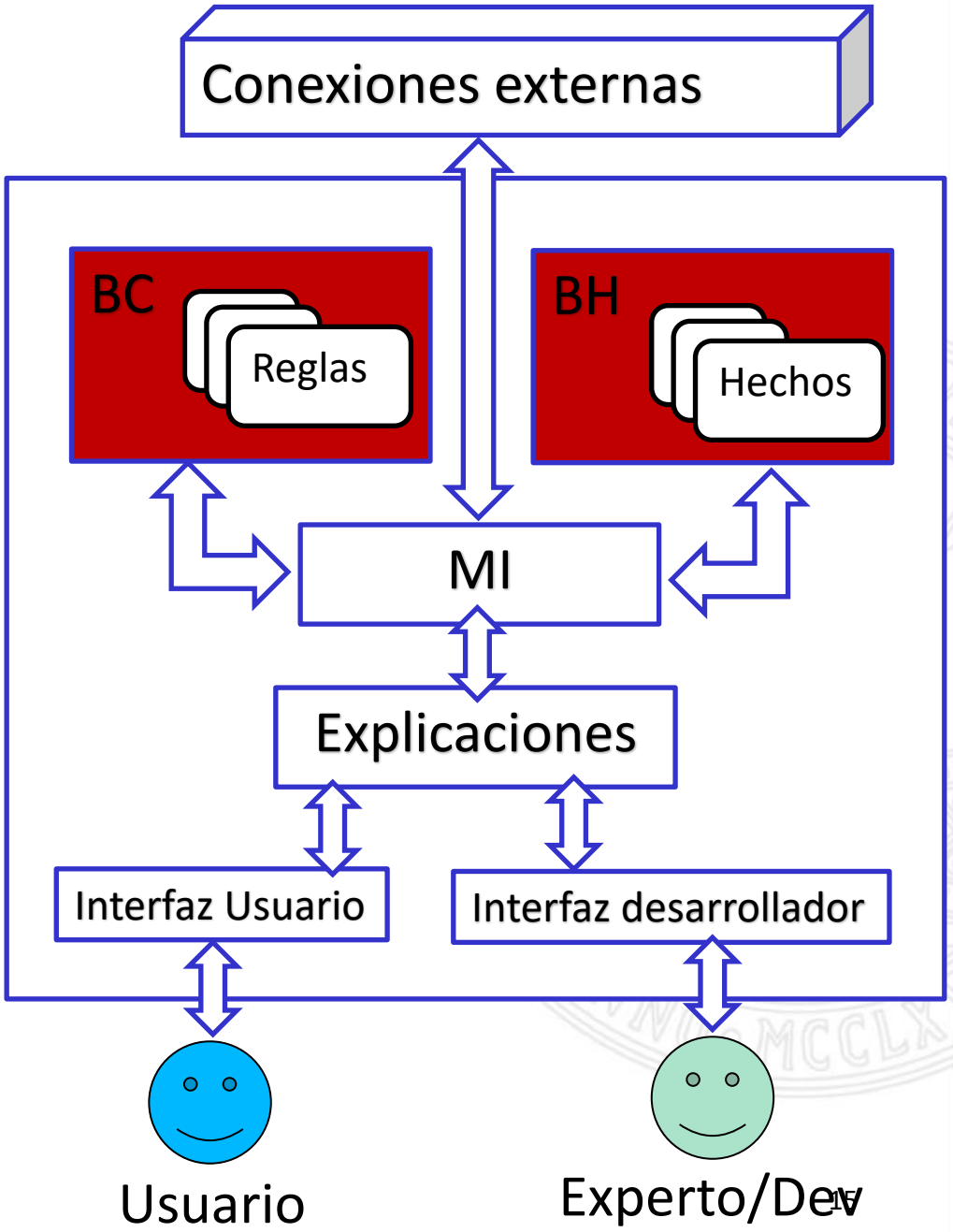
- Data base (DB)
- Almacena la información actual (estado) relativa al problema
- BH contiene al comienzo sólo hechos iniciales al problema
- Durante el proceso, se generan hechos inferidos a partir de esa información inicial, mediante la ejecución de las acciones de las reglas por parte del motor de inferencia (MI)



Unidad 2. Reglas en la minería de datos
Diferencia clave BC y BH

UNIVERSIDAD DE MURCIA

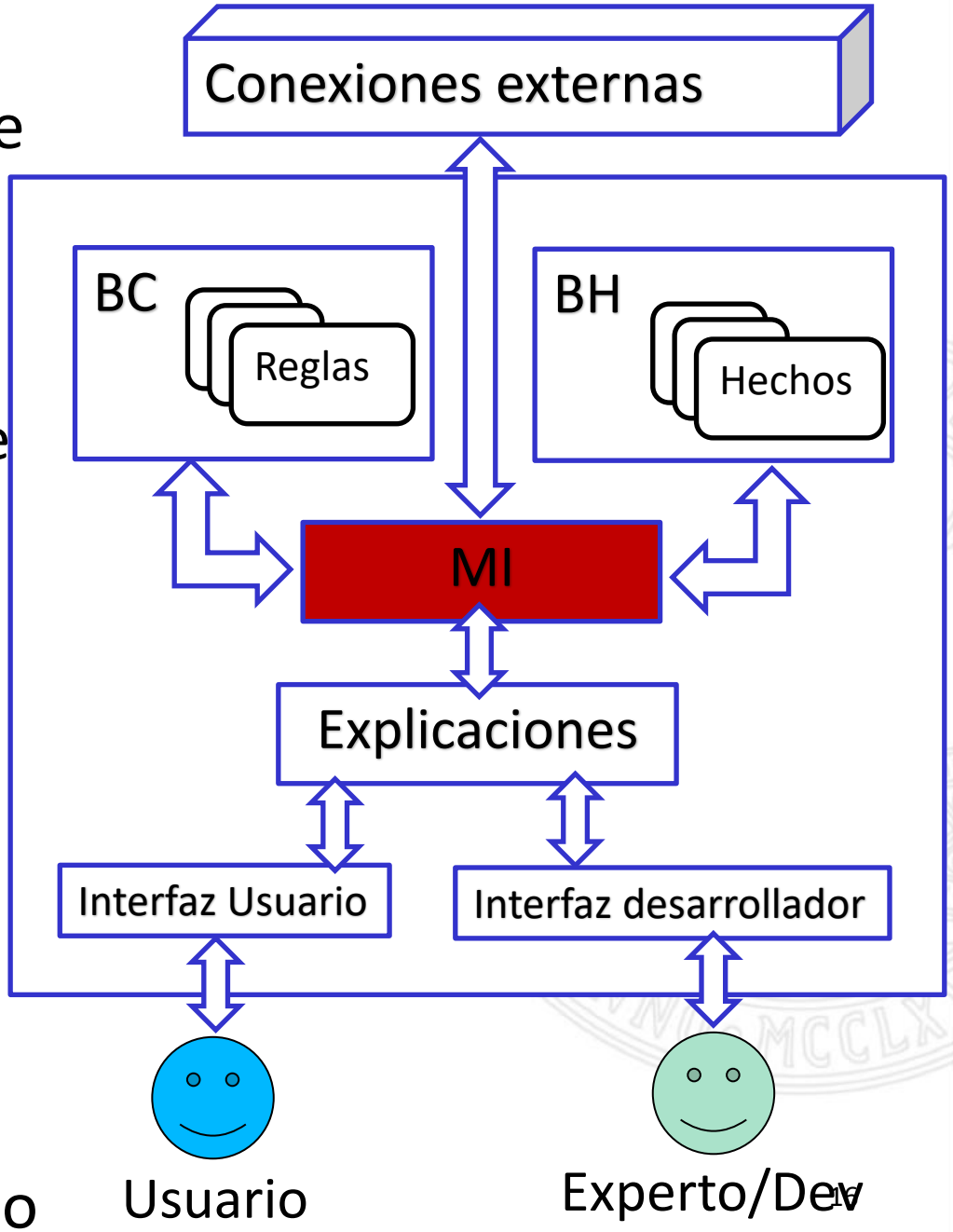
- Importante no confundir la BH y la BC
- La BC almacena conocimiento (estático, reglas) sobre como resolver el problema
- La BH almacena información (dinámica, hechos iniciales o inferidos) del problema
- Por ejemplo:
  - BC: Regla 1 {SI (fiebre y tos) ENTONCES COVID}
  - BH: Paciente Fernando tiene {fiebre y tos}



Unidad 2. Reglas en la minería de datos
Motor de inferencia (MI)

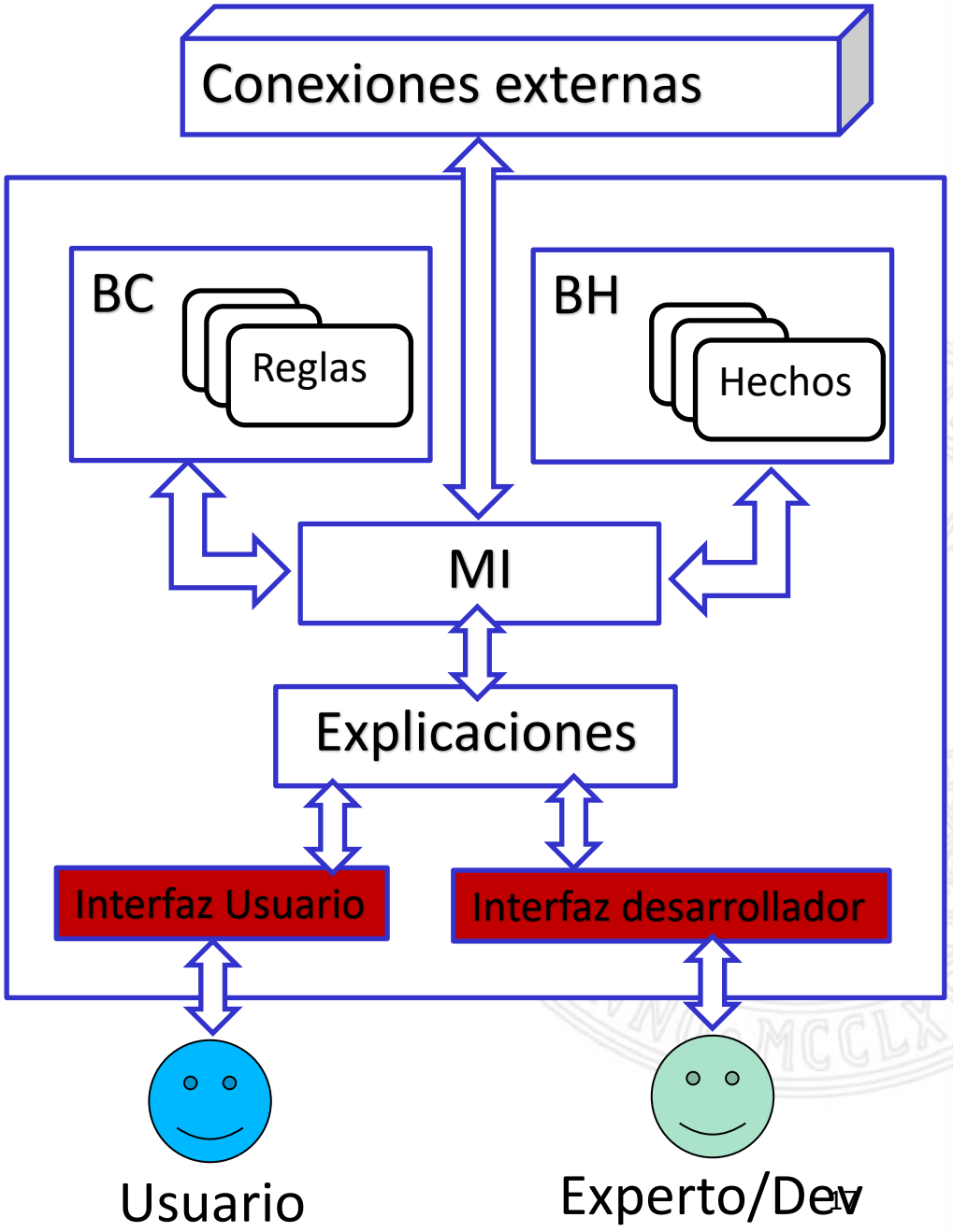
UNIVERSIDAD DE MURCIA

- Mecanismo para obtener conclusiones (hechos) a partir de BH y BC
- Dos técnicas de inferencia:
  - Encadenamiento hacia adelante (forward chaining)
  - Encadenamiento hacia atrás (backward chaining)
- Funciona como una caja negra
  - Input: BH y BC
  - Output: BH
- El usuario desconoce el proceso de inferencia, sólo ve el resultado



UNIVERSIDAD DE MURCIA

- Permite al usuario interactuar con el sistema
- Interacción con la BC:
  - Editar las reglas
  - Cuadro de mando reglas
- Interacción con la BH:
  - Introducir hechos iniciales
  - Consultar estado de la BH
- Interacción con MI
  - Configuración de parámetros
  - Seleccionar el tipo de estrategia/técnica de inferencia



UNIVERSIDAD DE
MURCIA

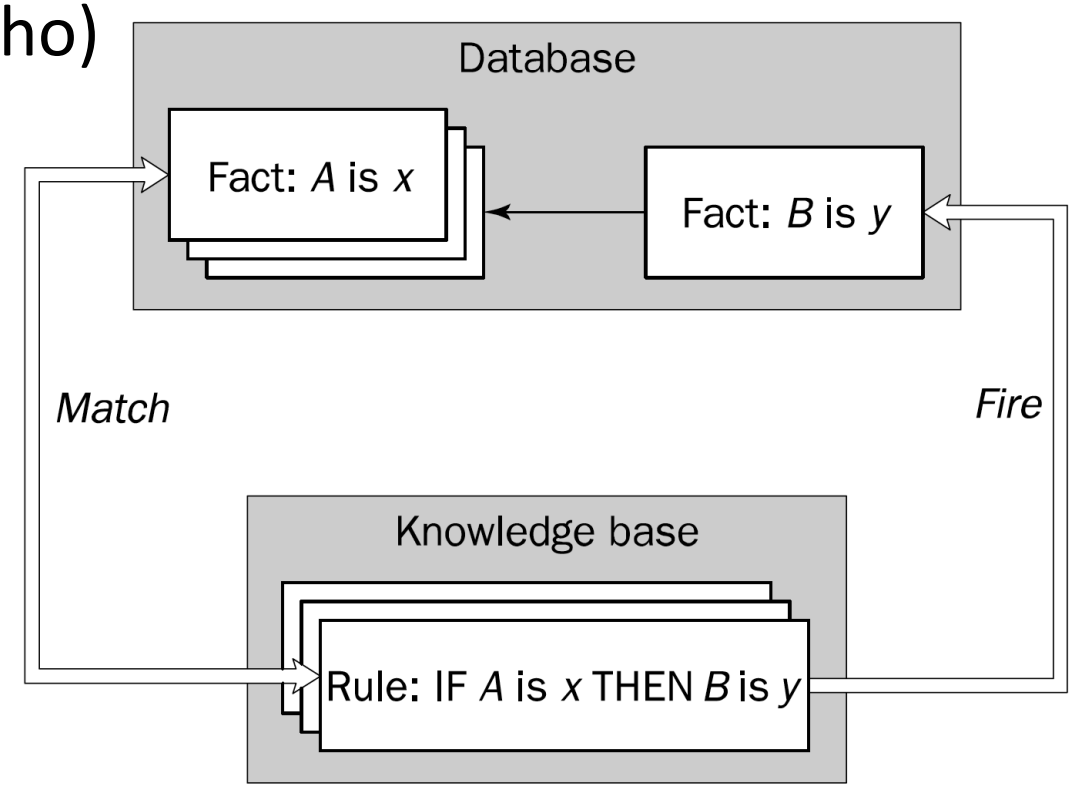
Unidad 2. Reglas en la minería de datos

INFERENCIA

Unidad 2. Reglas en la minería de datos
¿De qué se encarga el mecanismo de inferencia?

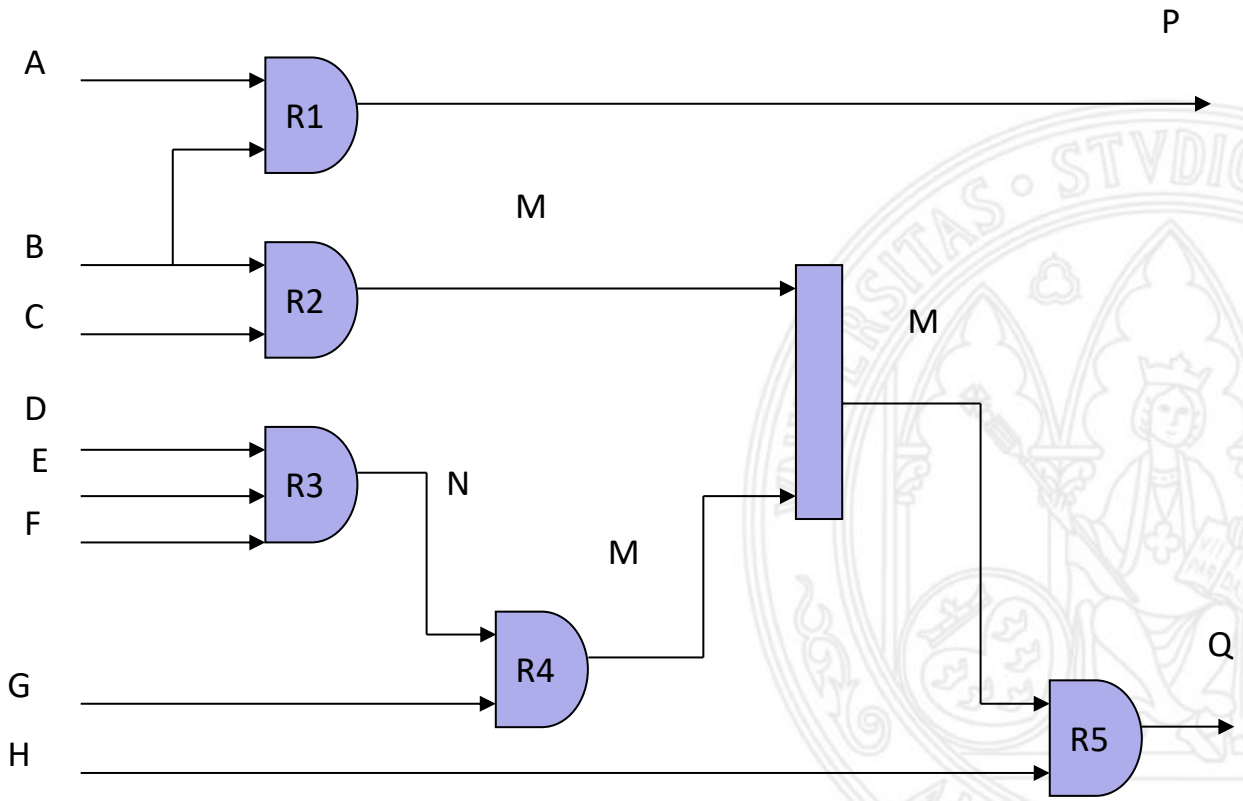
UNIVERSIDAD DE MURCIA

- El sistema de inferencia compara cada regla de la BC con los hechos de la BH
- Cuando una condición *IF (condición)* se cumple, entonces la regla dispara su THEN (acción)
- Este proceso de equiparación (matching) genera una cadena de inferencia (hecho)



- El sistema de reglas se puede representar mediante un circuito de puertas AND

KNOWLEDGE BASE
R1: A & B → P
R2: B & C → M
R3: D&E&F → N
R4: N&G → M
R5: H&M → Q



Unidad 2. Reglas en la minería de datos
Ejemplo 2

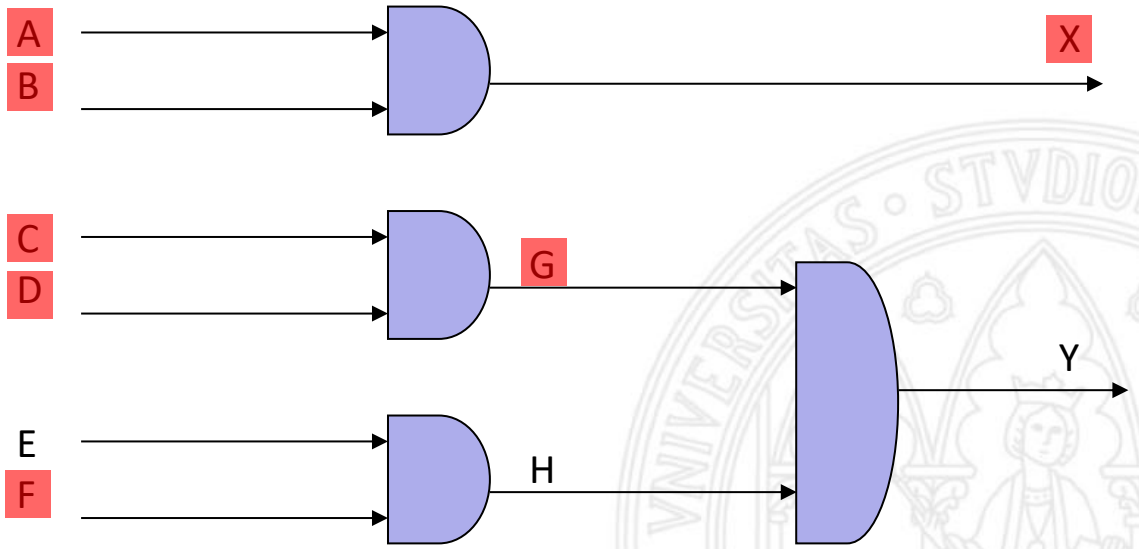
UNIVERSIDAD DE MURCIA

- Podemos ver como se activa el circuito en base a la BH inicial

BC:

R1: SI A & B ENTONCES X
R2: SI C & D ENTONCES G
R3: SI E & F ENTONCES H
R4: SI G & H ENTONCES Y

BH Inicial = {A, B, C, D, F}



Reglas activadas (conjunto de conflicto): R1, R2
BH Final = {A, B, C, D, F, X, G}

- Base de conocimiento (**BC**) y base de hechos (**BH**)
- **Objetivo**: meta que se establece en el sistema experto
- **Problema de equiparación**: encontrar las reglas que se tendrían que activar
- **Conjunto conflicto**: conjunto de reglas que podrían ser ejecutadas al haber sido activadas
- **Resolución de conflicto**: elegir una regla del conjunto conflicto para ser ejecutada
- **Condición de parada**: criterio para detener el proceso de inferencia

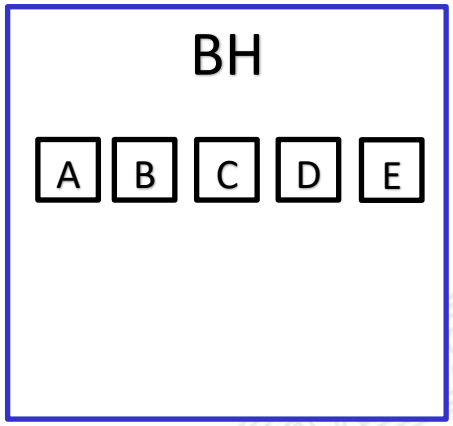
- Conocido como razonamiento dirigido por datos
- Comienza con los hechos de la BH y es un proceso constructivo
- Conocimiento del dominio: reglas de producción
  - Las reglas SI-ENTONCES se pueden entender como:
    - "cuando LHS se cumple con hechos (hace match), entonces RHS se ejecuta
  - Muchas de las reglas pueden no tener que ver con la meta final
- Cuando se tienen cientos de reglas y una sola meta final, puede que encadenamiento hacia atrás sea más apropiado

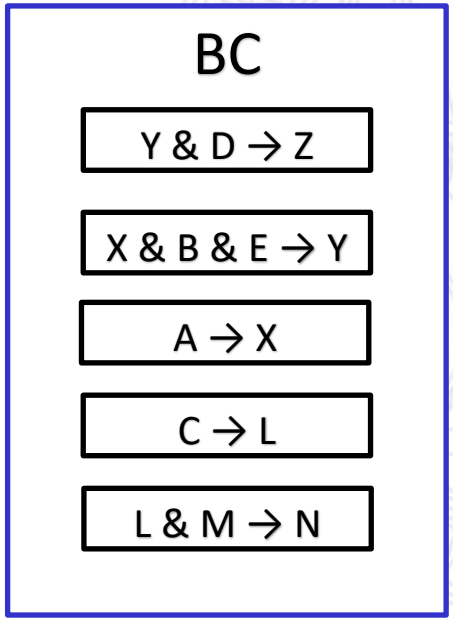
Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia delante

Meta/Objetivo: Z

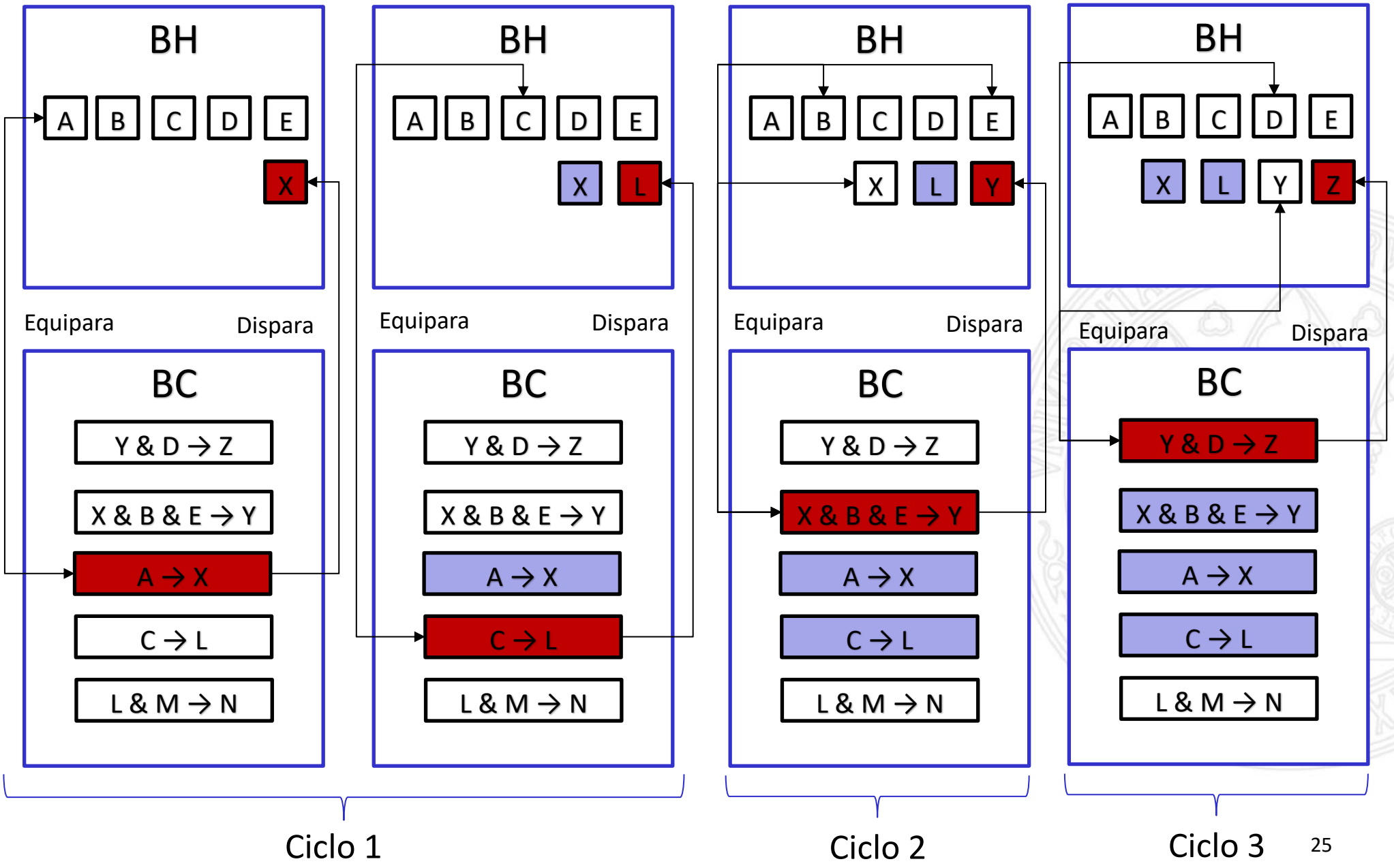
Base de hechos inicial:
{A, B, C, D, E}

Base de conocimiento:
R1: Y & D → Z
R2: X & B & E → Y
R3: A → X
R4: C → L
R5: L & M → N





Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia delante resuelto

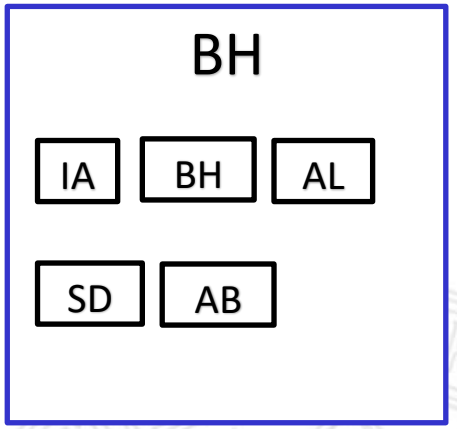


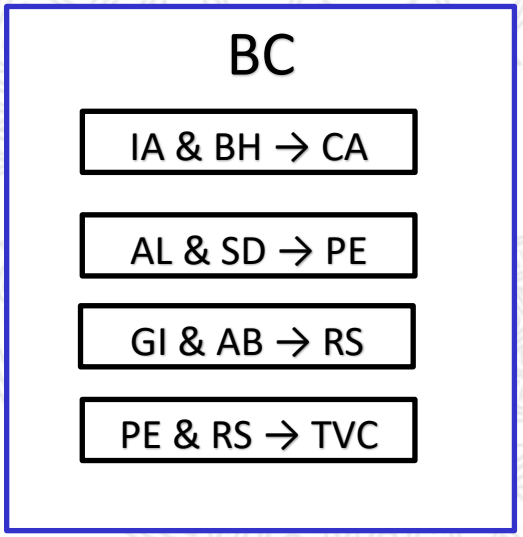
Ejercicio 1: Aprobación de Créditos y Tarjetas Bancarias

Meta/Objetivo: TarjetaVipConcedida

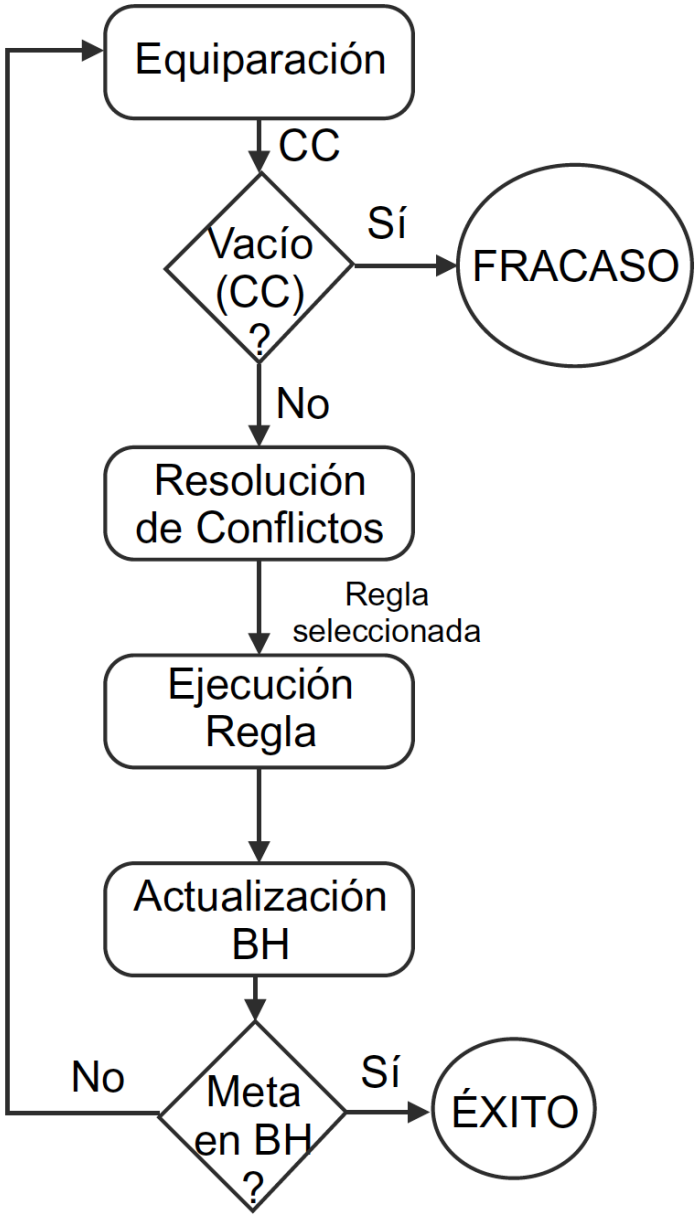
Base de hechos inicial:
{IngresosAltos, BuenHistorial, AntiguedadLaboral, SinDeudas, AvalBancario}

Base de conocimiento:
R1: IngresosAltos & BuenHistorial → CreditoAprobado
R2: AntiguedadLaboral & SinDeudas → PerfilEstable
R3: GarantiaInmobiliaria & AvalBancario → RespaldoSolido
R4: PerfilEstable & RespaldoSolido → TarjetaVipConcedida





- Diagrama de flujo (*flowchart* o esquema algorítmico)
- Representa visualmente el **ciclo de inferencia iterativo** (o ciclo de ejecución) del algoritmo de encadenamiento hacia delante



Unidad 2. Reglas en la minería de datos
Algoritmo de encadenamiento hacia delante

UNIVERSIDAD DE MURCIA

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:         Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo de encadenamiento hacia delante

UNIVERSIDAD DE MURCIA

BC:

R1: b & d & e → f

R2: d & g → a

R3: c & f → a

R4: b → x

R5: d → e

R6: a & x → h

R7: c → d

R8: x & c → a

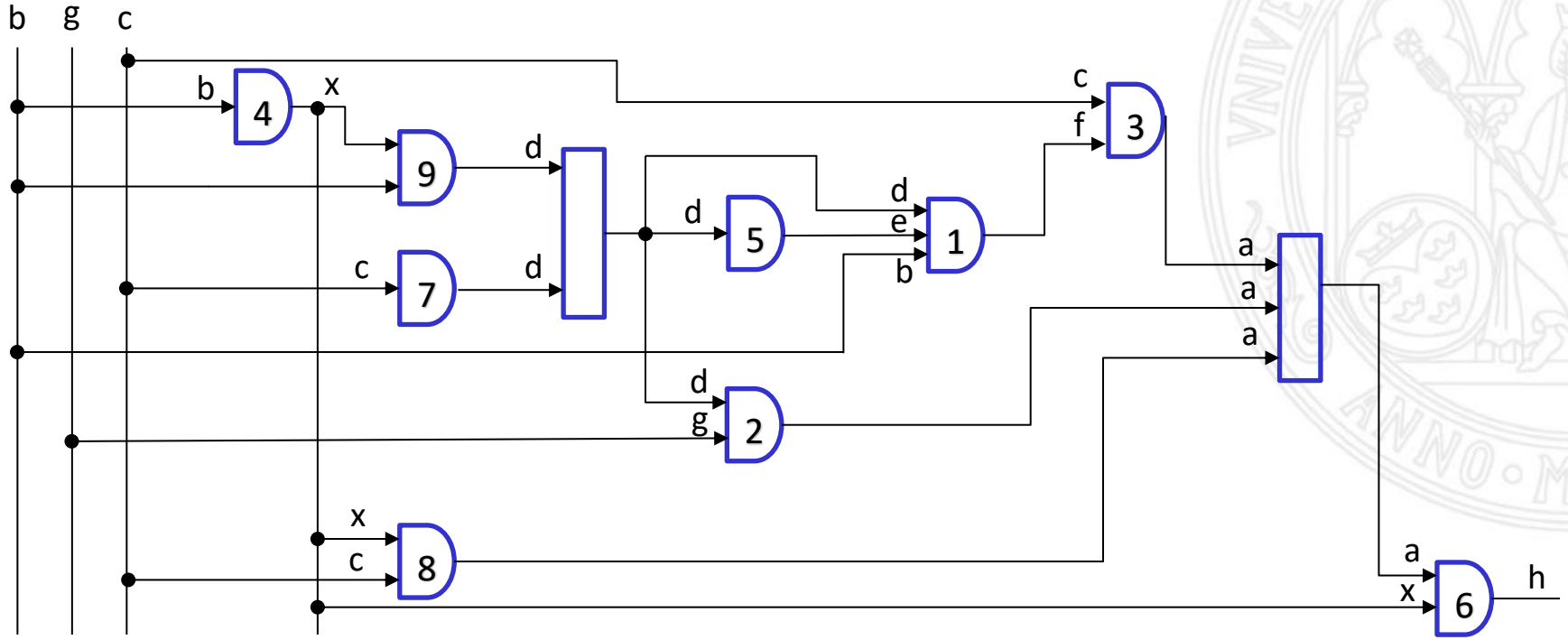
R9: x & b → d

BH inicial:

{b, c}

Meta a alcanzar: *h*

Suponemos una vez disparada una regla, ya no vuelve a disparar



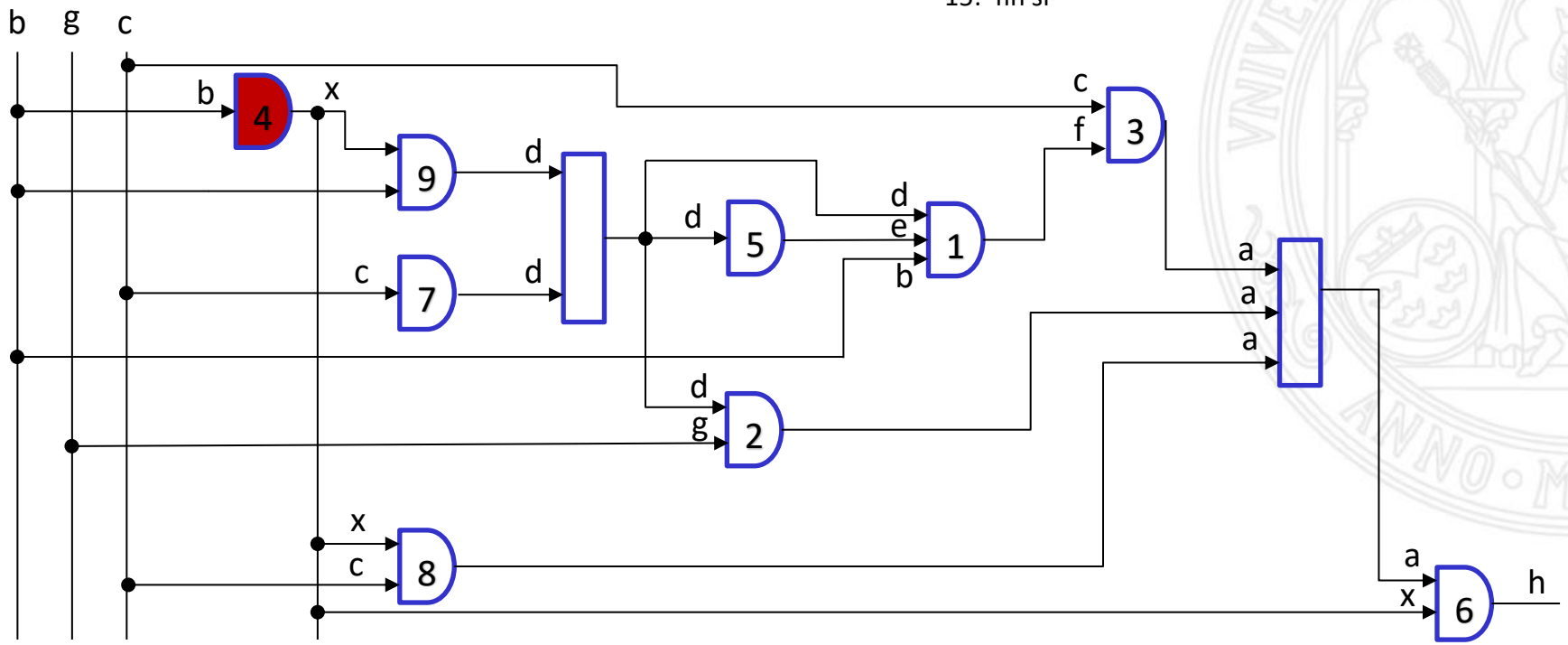
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 1

UNIVERSIDAD DE MURCIA

1. BH = {b, c}
2. ConjuntoConflicto= {4, 7}
3. R = 4 // Resolución conflicto por orden "b → x"
4. Aplicar (R, BH)
5. BH = {b, c, x}
6. Marca (R)

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:         Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



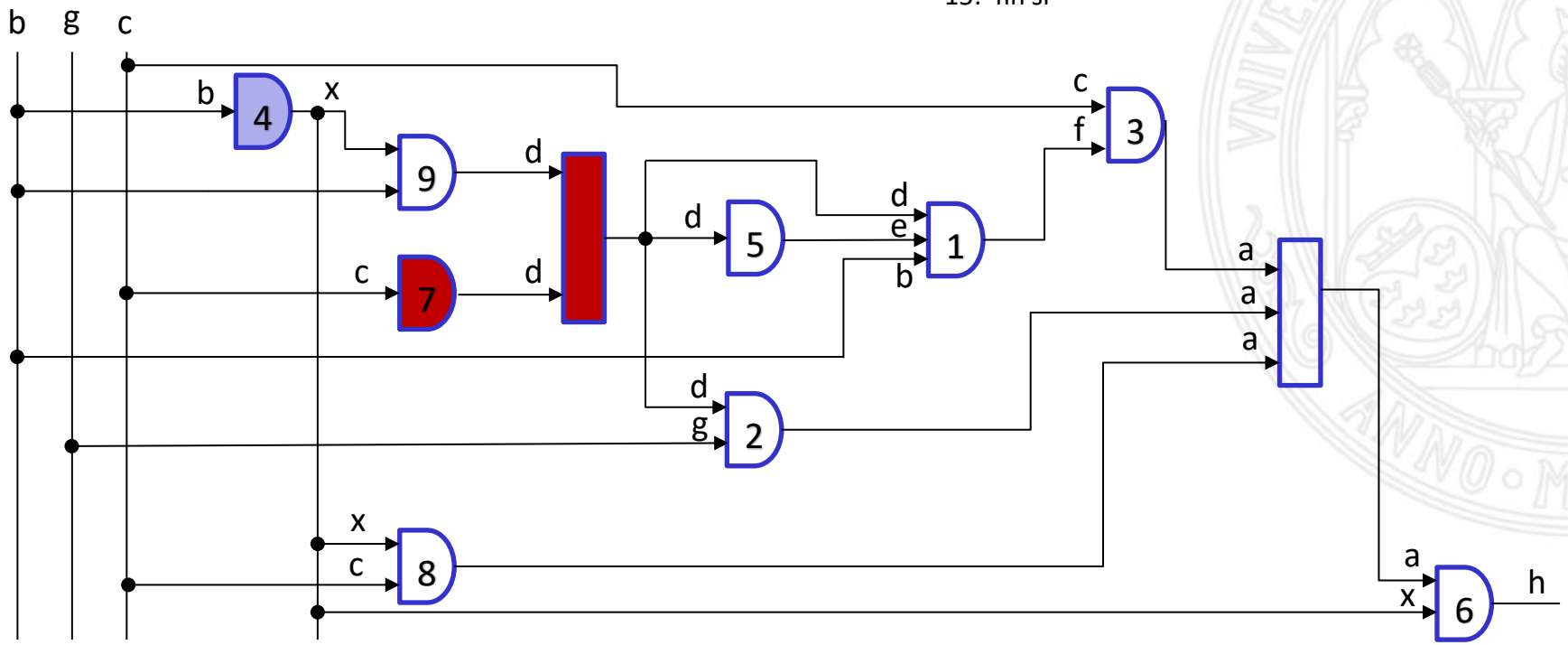
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 2

UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto = {7, 8, 9}
2. R = 7 // Resolución conflicto por orden "c → d"
3. Aplicar (R, BH)
4. BH = {b, c, x, d}
5. Marca (R)

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:     ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:     si NoVacío(ConjuntoConflicto) entonces
5:        R = Resolver(ConjuntoConflicto);
6:        NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:        Actualizar(BH, NuevosHechos);
8:        Marca(R, BC)
9:     si no // else en lenguajes de programación
10:          Fin = Verdadero;
11:    fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:    devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



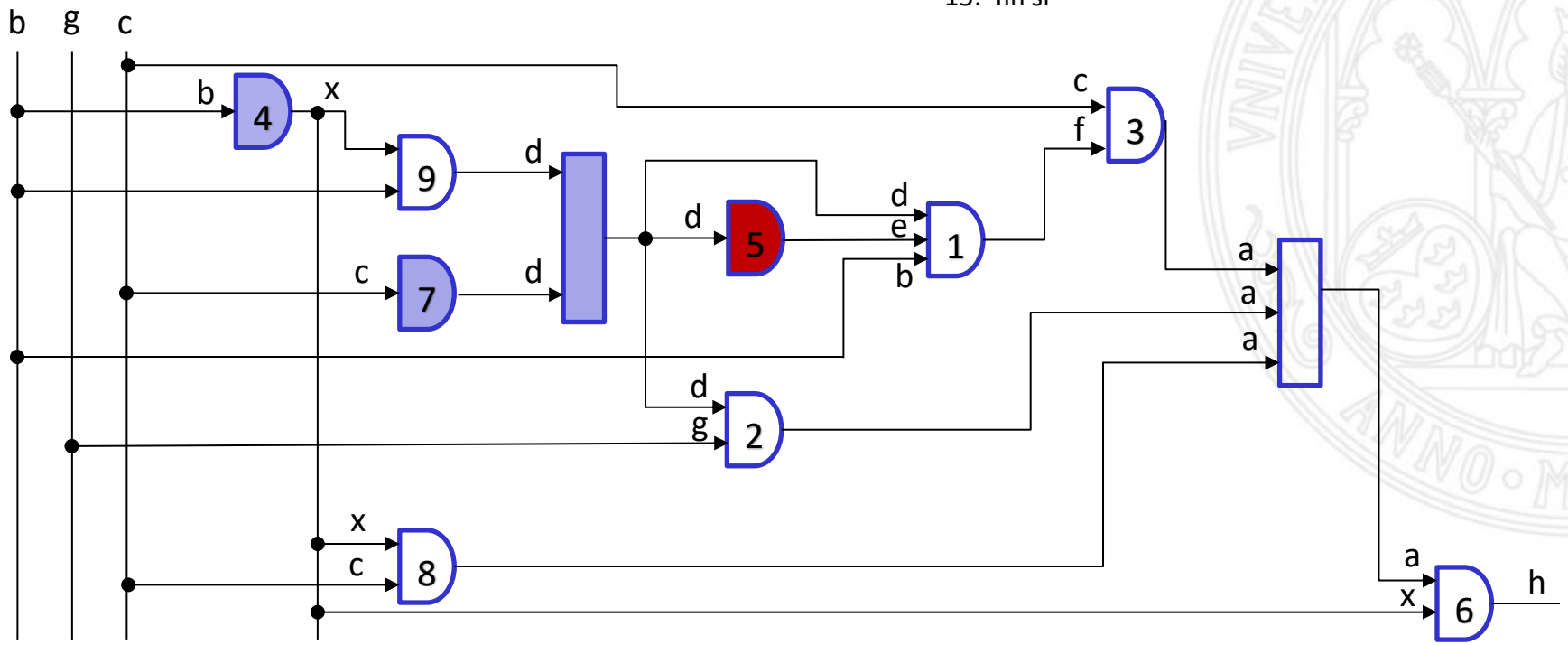
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 3

UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto = {5, 8, 9}
2. R = 5 // Resolución conflicto por orden "d → e"
3. Aplicar (R, BH)
4. BH = {b, c, x, d, e}
5. Marca (R)

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:             Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



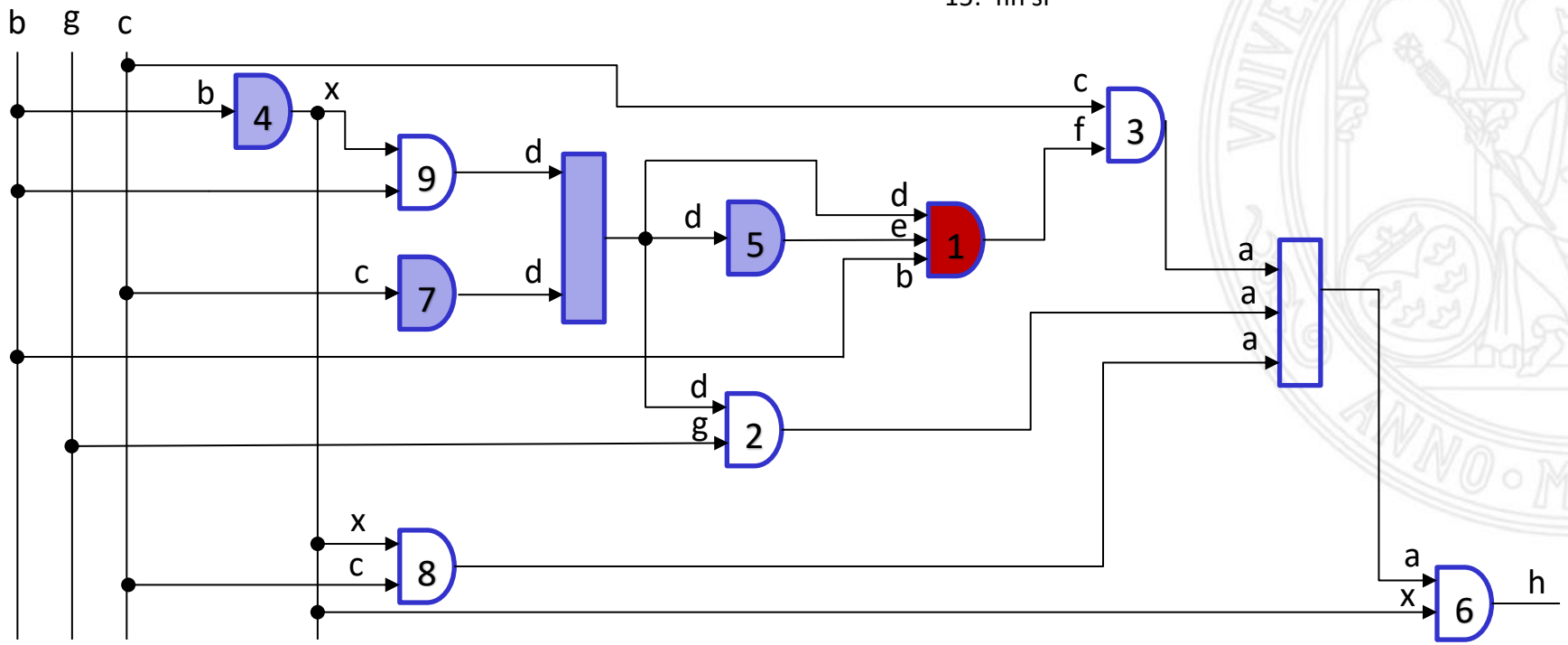
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 4

UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto = {1, 8, 9}
2. R = 1 // Resolución conflicto por orden "b & d & e → f"
3. Aplicar (R, BH)
4. BH = {b, c, x, d, e, f}
5. Marca (R)

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:             Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



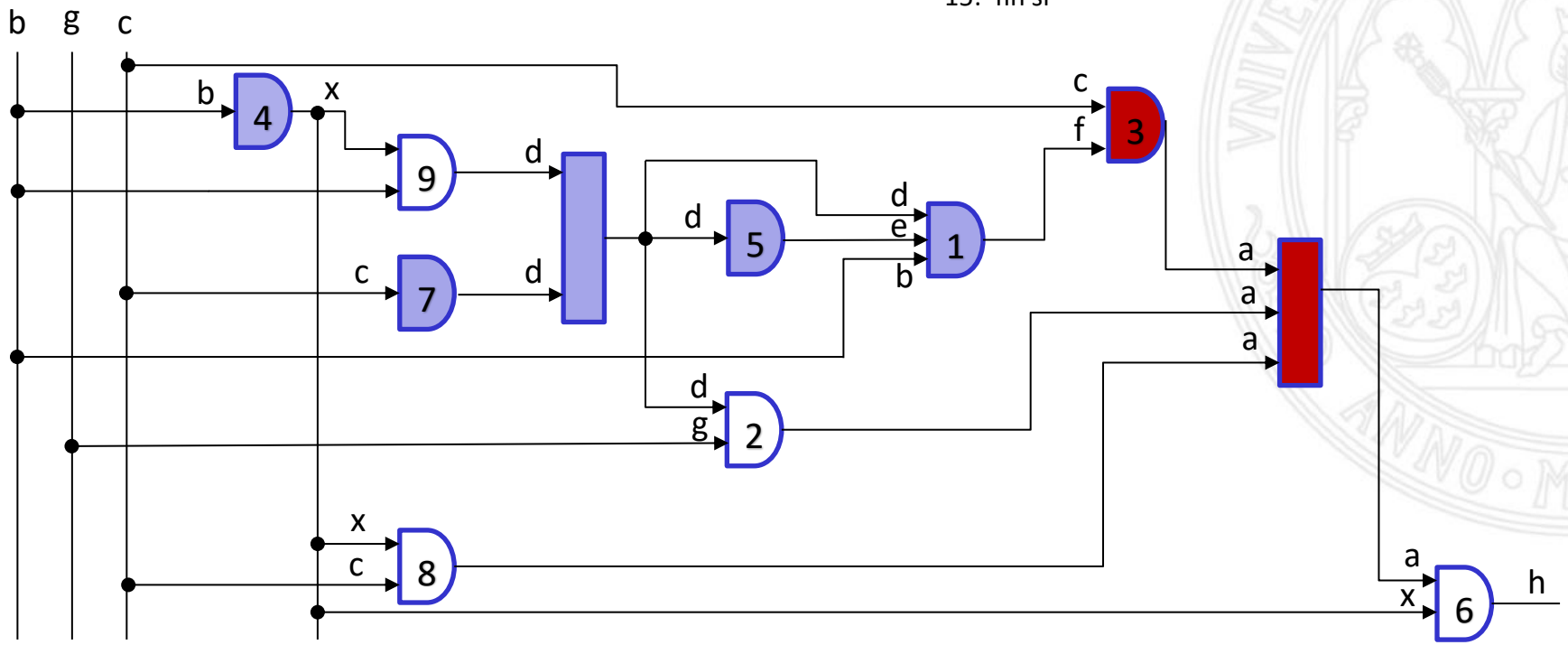
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 5

UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto = {3, 8, 9}
2. R = 3 // Resolución conflicto por orden "c & f → a"
3. Aplicar (R, BH)
4. BH = {b, c, x, d, e, f, a}
5. Marca (R)

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:             Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



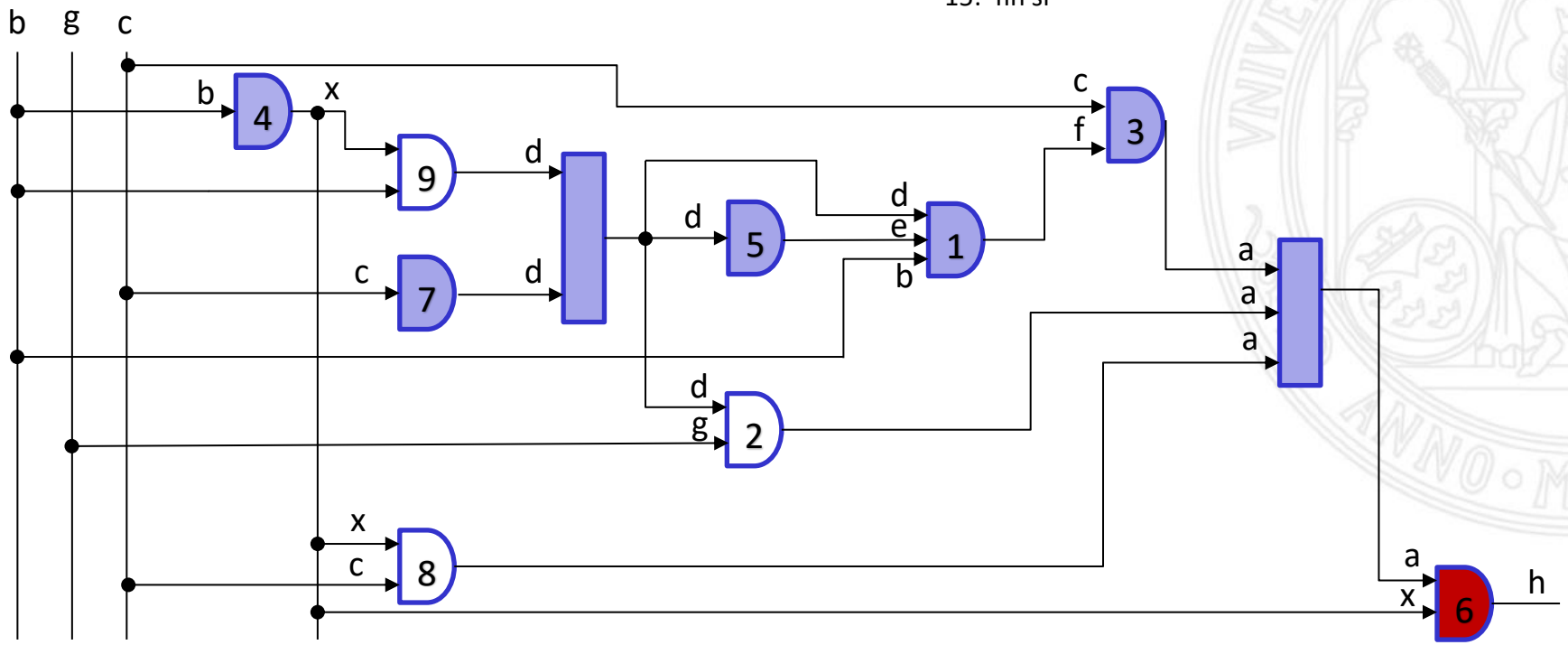
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia delante: Ciclo 6

UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto = {6, 8, 9}
2. R = 6 // Resolución conflicto por orden "a & x → h"
3. Aplicar (R, BH)
4. BH = {b, c, x, d, e, f, a, h}
5. Marca (R)
6. Devuelve "éxito"

```
1:  BH = HechosIniciales, Fin = Falso;
2:  mientras NoContenida(Meta, BH) y No(Fin) hacer
3:      ConjuntoConflicto = Equiparar(Antecedente(BC), BH);
4:      si NoVacío(ConjuntoConflicto) entonces
5:          R = Resolver(ConjuntoConflicto);
6:          NuevosHechos = Aplicar(R,BH); // ejecutar consecuente
7:          Actualizar(BH, NuevosHechos);
8:          Marca(R, BC)
9:      si no // else en lenguajes de programación
10:             Fin = Verdadero;
11:     fin si
12: fin mientras
13: si Contenida(Meta, BH) entonces
14:     devolver "éxito";
15: fin si
```

Meta a alcanzar: *h*



Unidad 2. Reglas en la minería de datos
Comparación criterios de resolución de conflictos

UNIVERSIDAD DE MURCIA

- El criterio de resolución de conflictos es clave para el comportamiento del sistema
- Previamente hemos usado criterio basado en número de regla
- Se podría establecer un criterio que aplique la que tenga clausulas antecedente más restrictivas
  - ¿Tiene sentido?
- Se pueden aplicar otros criterios, por ejemplo, el desarrollador establece prioridades

Criterio 1: Número de regla

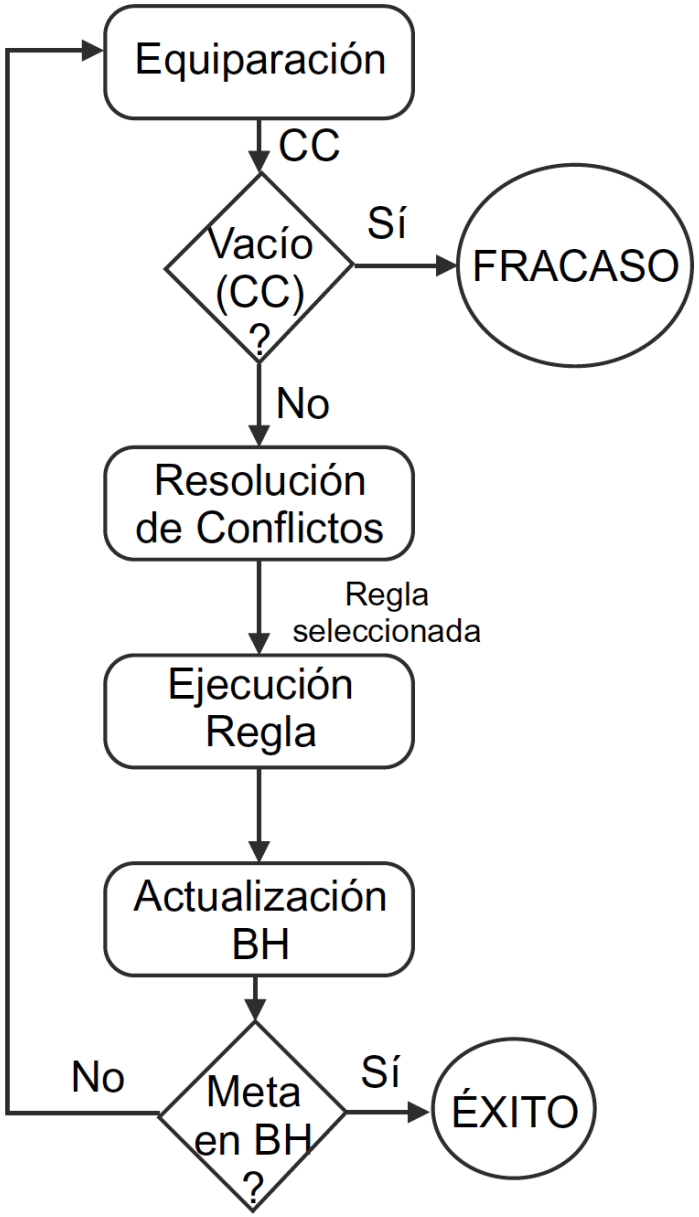
Ciclo	BH	Conjunto Conflicto	Resolución
1	$\{b, c\}$	$\{4, 7\}$	4
2	$\{x, b, c\}$	$\{7, 8, 9\}$	7
3	$\{d, x, b, c\}$	$\{5, 8, 9\}$	5
4	$\{e, d, x, b, c\}$	$\{1, 8, 9\}$	1
5	$\{f, e, d, x, b, c\}$	$\{3, 8, 9\}$	3
6	$\{a, f, e, d, x, b, c\}$	$\{6, 8, 9\}$	6

Criterio 2: Mayor número de clausulas antecedente

Ciclo	BH	Conjunto Conflicto	Resolución
1	$\{b, c\}$	$\{4, 7\}$	4
2	$\{x, b, c\}$	$\{7, 8, 9\}$	8
3	$\{a, x, b, c\}$	$\{6, 7, 9\}$	6

Unidad 2. Reglas en la minería de datos
Resumen encadenamiento hacia delante

- El ciclo es relativamente sencillo, pero se complica una vez se añaden variables y cuantificadores en las reglas
- Los inconvenientes son:
  - Fase resolución de conflictos es crítica
  - El disparo de las reglas elegidas puede ser inútil, alcanzando conclusiones que no tienen interés (explosión combinatoria)
  - Se necesita cargar la BH con todas las afirmaciones sin saber si son útiles o faltan datos



Unidad 2. Reglas en la minería de datos
Ejercicio 2: Algoritmo hacia delante

UNIVERSIDAD DE MURCIA

BC:

R1: p & r & s → u
R2: r & v → w
R3: q & u → w
R4: p → t
R5: r → s
R6: w & t → z
R7: q → r
R8: t & q → w
R9: t & p → r

BH inicial:
{p, q}
Meta a alcanzar: *z*

Suponemos una vez disparada una regla, ya no vuelve a disparar

- Genera la tabla resumen de los ciclos (como el de la transparencia 36)
- Genera el grafico de puertas lógicas.

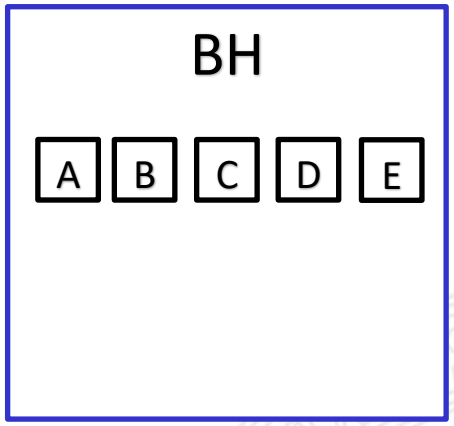
- Conocido como razonamiento dirigido por objetivos
- El sistema tiene un objetivo (solución hipótesis) y la inferencia intenta demostrar que la hipótesis se cumple
- Conocimiento del dominio: reglas
  - Las reglas SI-ENTONCES se pueden entender como:
    - "la RHS describe objetivos (objetivo final y objetivos intermedios)"
- Hay que encontrar las LHS que hagan equiparación (match) con la BH para alcanzar esos objetivos
  - Normalmente no tendrá ese hecho
- El proceso de inferencia sigue acumulando reglas hasta que no encuentra una regla capaz de probar el objetivo (final/intermedio)

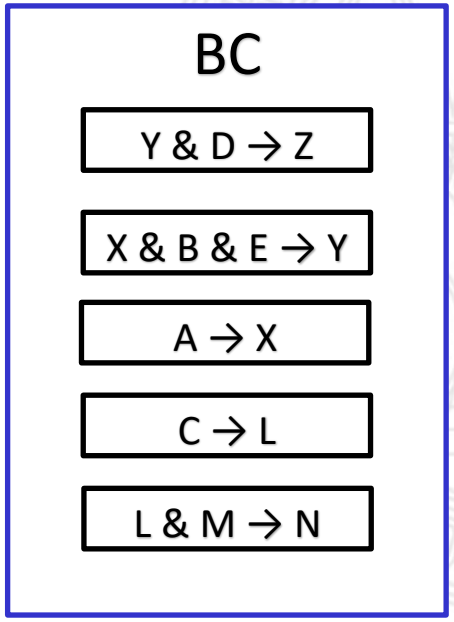
Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia atrás

Base de hechos inicial:
{A, B, C, D, E}

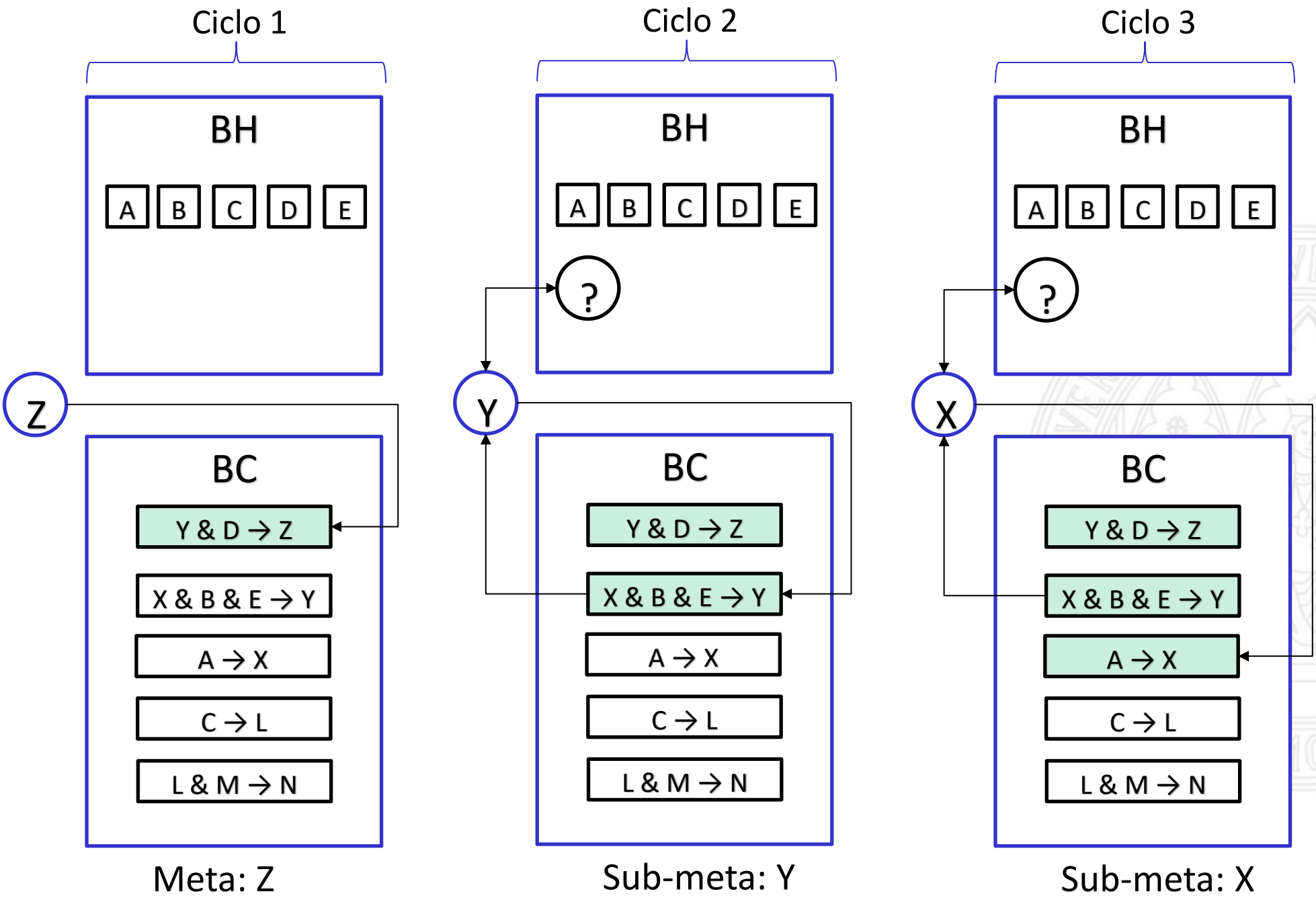
Meta = Z

Base de conocimiento:
R1: Y & D → Z
R2: X & B & E → Y
R3: A → X
R4: C → L
R5: L & M → N

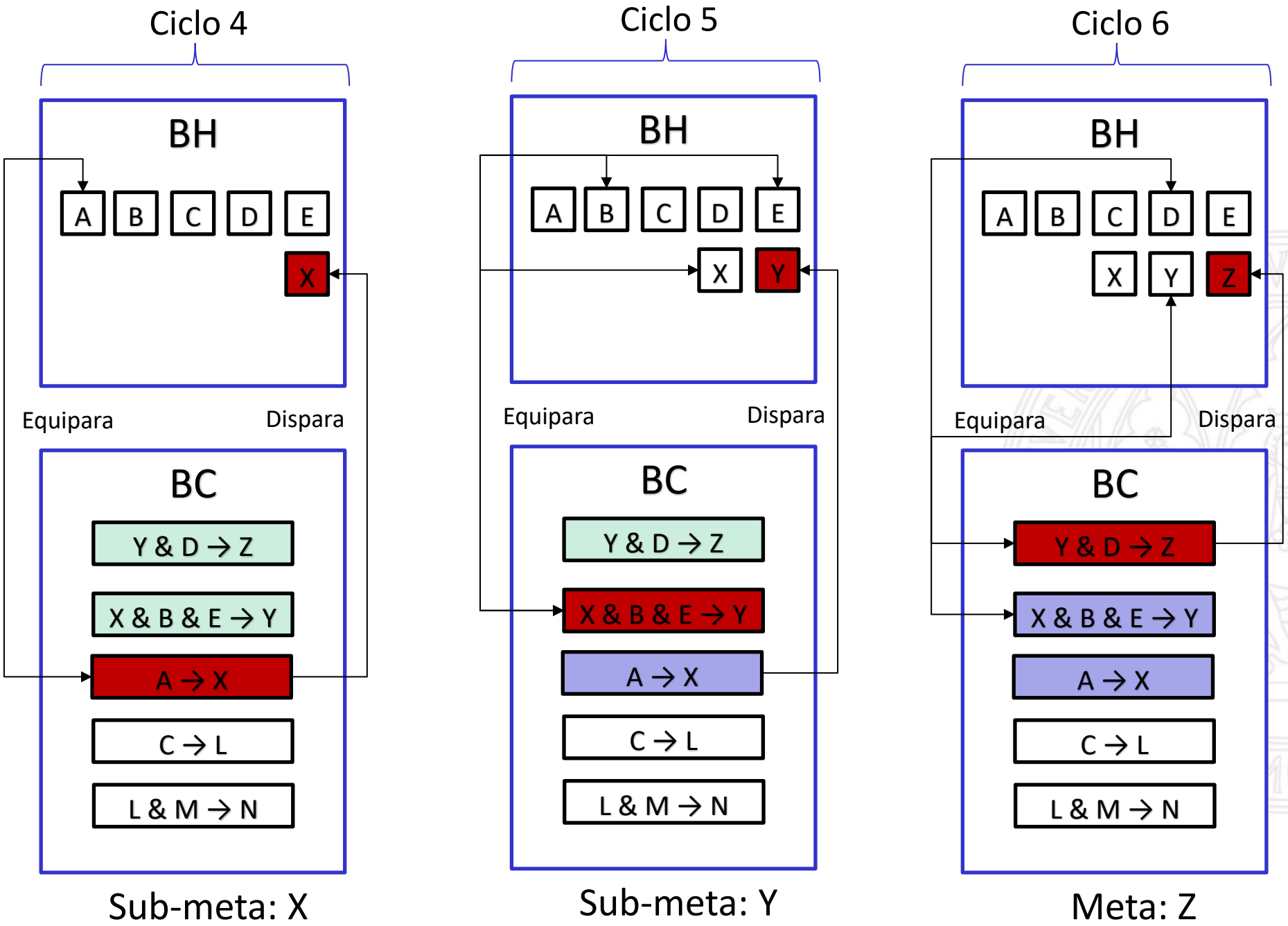




Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia atrás



Unidad 2. Reglas en la minería de datos
Ejemplo encadenamiento hacia atrás



Verificar(Meta):

```
 1: Verificado = Falso;
 2: si Contenida(Meta, BH) entonces
 3:     devolver Verdadero;
 4: si no
 5:     ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
 6:     mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
 7:         R = Resolver(ConjuntoConflicto);
 8:         Eliminar(R, ConjuntoConflicto);
 9:         NuevasMetas = ExtraerAntecedentes(R);
10:         Verificado = Verdadero;
11:         mientras NoVacío(NuevasMetas) y Verificado hacer
12:             MetaActual = SeleccionarMeta(NuevasMetas);
13:             Eliminar(MetaActual, NuevasMetas);
14:             Verificado = Verificar(MetaActual, BH);
15:         fin mientras
16:         si Verificado entonces
17:             Añadir(Meta, BH);
18:         fin si
19:     fin mientras
20:     devolver Verificado;
21: fin si
```

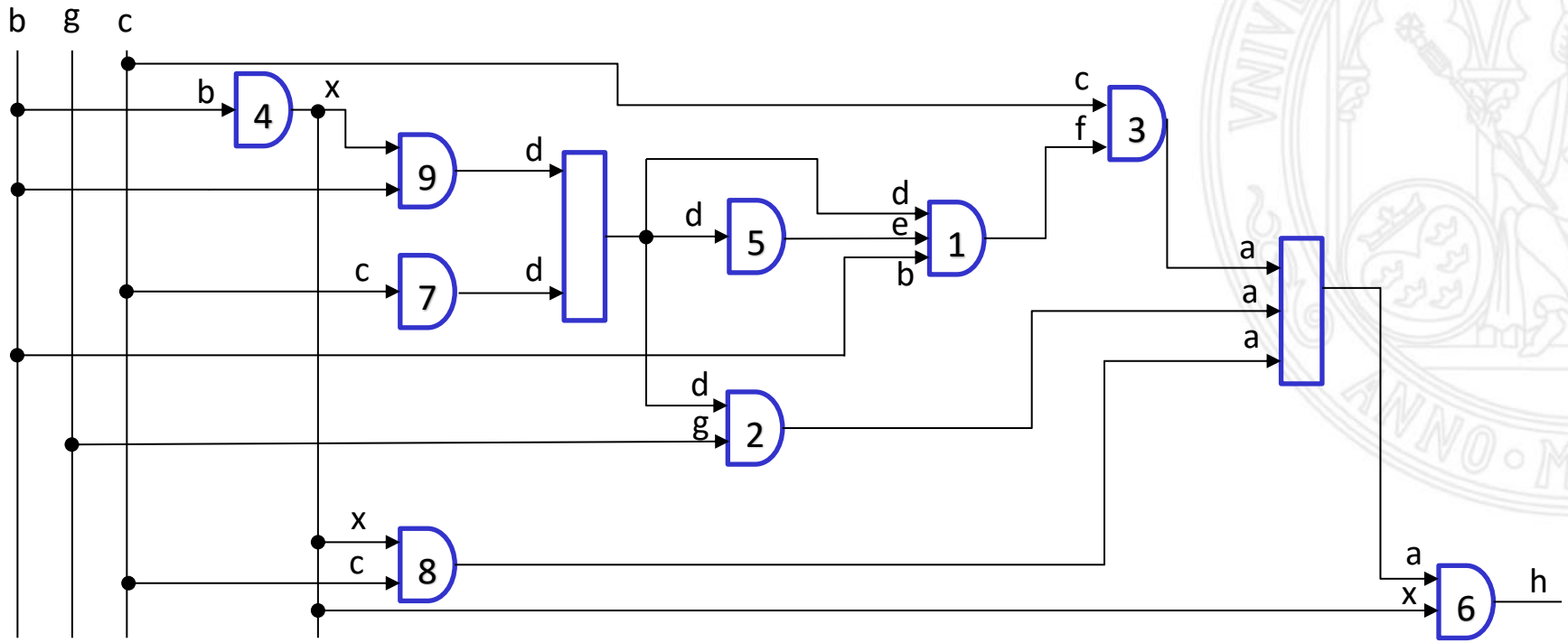
UNIVERSIDAD DE MURCIA

BC:
R1: b & d & e → f
R2: d & g → a
R3: c & f → a
R4: b → x
R5: d → e

BC:
R6: a & x → h
R7: c → d
R8: x & c → a
R9: x & b → d

BH inicial:
{b, c}
Meta a alcanzar: *h*

Suponemos una vez disparada una regla, ya no vuelve a disparar
Criterio resolución: Menor número



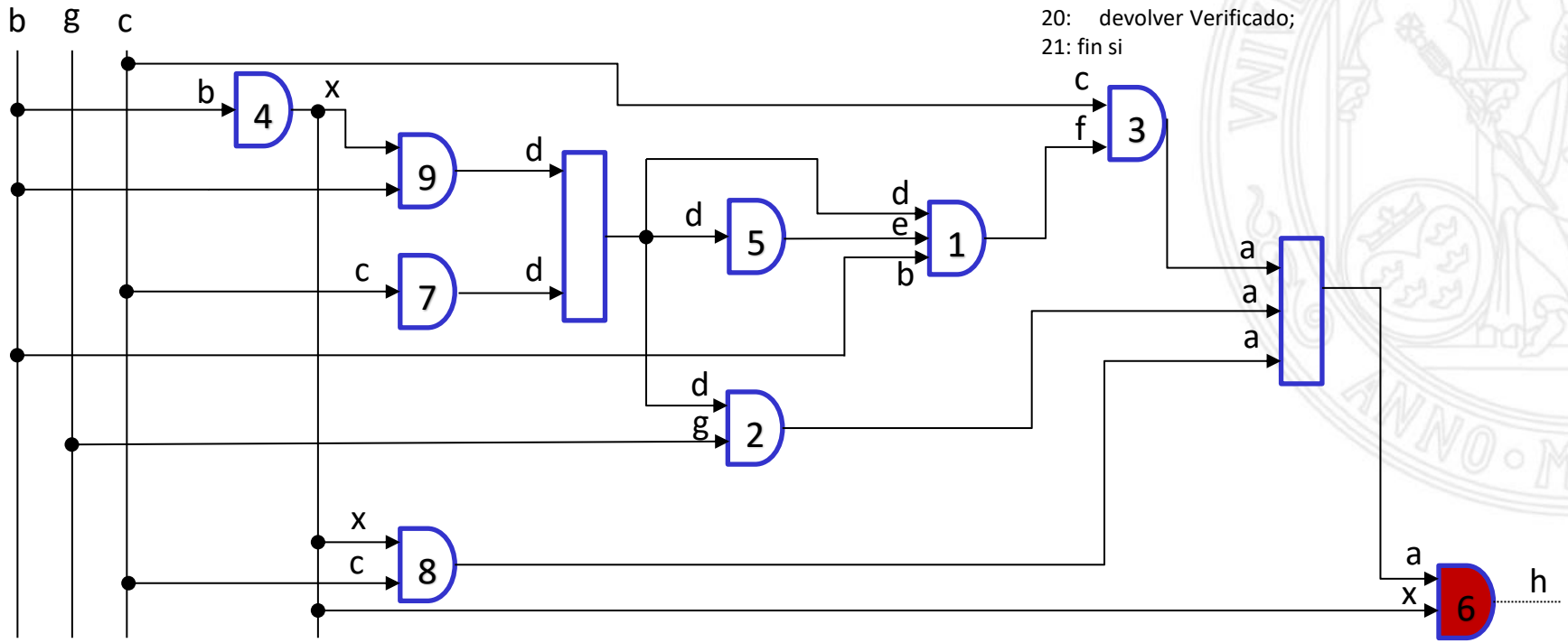
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 1

UNIVERSIDAD DE MURCIA

1: Meta = h
2. ConjuntoConflicto= {6}
3. R = 6 // Resolución conflicto por orden "a & x → h"
4. Eliminar (R, CC) → ConjuntoConflicto= {}
5. NuevasMetas= {a, x}
6: MetaActual = a
7: Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {x}
8: Verificar(MetaActual, BH) // No verificado, llamada recursiva

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:      Verificado = Verdadero;
11:      mientras NoVacío(NuevasMetas) y Verificado hacer
12:         MetaActual = SeleccionarMeta(NuevasMetas);
13:         Eliminar(MetaActual, NuevasMetas);
14:         Verificado = Verificar(MetaActual, BH);
15:      fin mientras
16:      si Verificado entonces
17:         Añadir(Meta, BH);
18:      fin si
19:   fin mientras
20:   devolver Verificado;
21: fin si
```

Meta a alcanzar: *h*



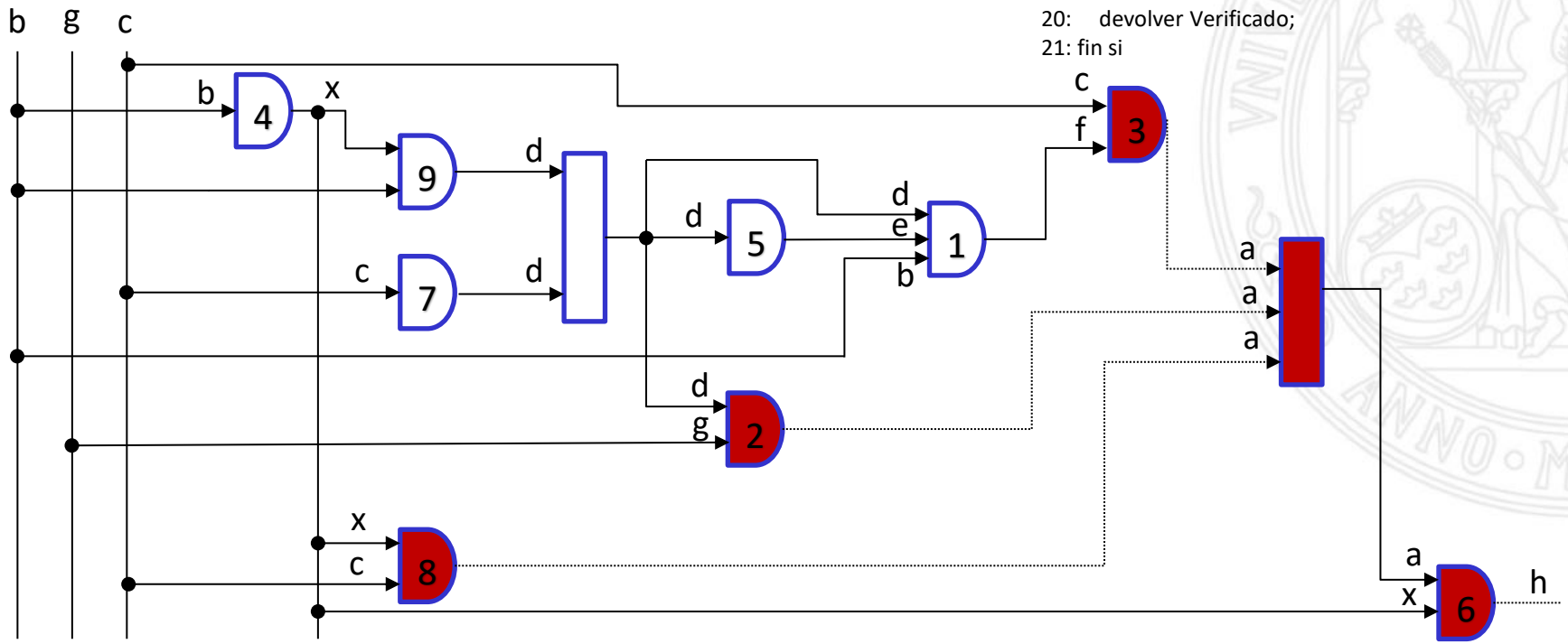
Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 2

UNIVERSIDAD DE MURCIA

1. Meta = a
2. ConjuntoConflicto= {2, 3, 8}
3. R = 2 // Resolución conflicto por orden "d & g → a"
4. Eliminar (R, CC) → ConjuntoConflicto= {3, 8}
5. NuevasMetas= {d, g}
6. MetaActual = d
7. Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {g}
8. Verificar(MetaActual, BH) // No verificado, llamada recursiva

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:       Verificado = Verdadero;
11:       mientras NoVacío(NuevasMetas) y Verificado hacer
12:          MetaActual = SeleccionarMeta(NuevasMetas);
13:          Eliminar(MetaActual, NuevasMetas);
14:          Verificado = Verificar(MetaActual, BH);
15:       fin mientras
16:       si Verificado entonces
17:          Añadir(Meta, BH);
18:       fin si
19:    fin mientras
20:    devolver Verificado;
21: fin si
```

Meta a alcanzar: *a*



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 3

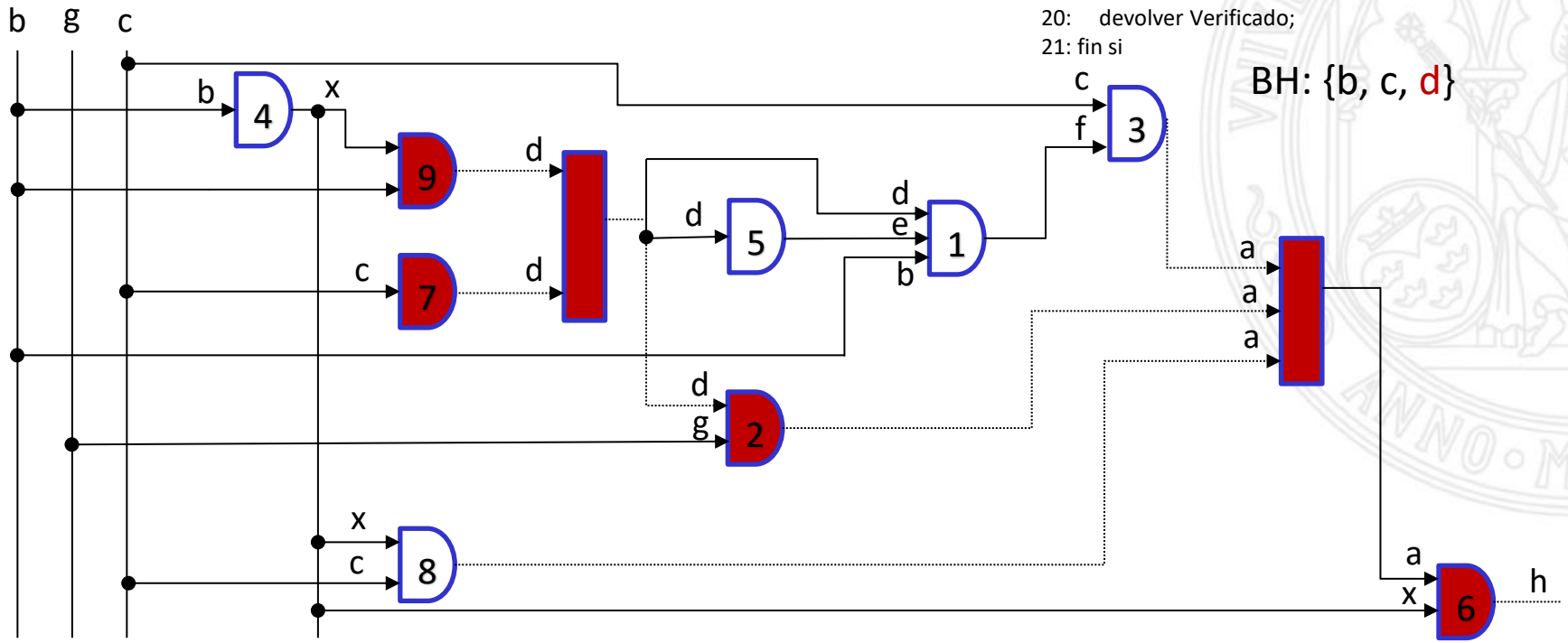
UNIVERSIDAD DE MURCIA

1. Meta = d
2. ConjuntoConflicto= {7, 9}
3. R = 7 // Resolución conflicto por orden "c → d"
4. Eliminar (R, CC) → ConjuntoConflicto= {9}
5. NuevasMetas= {c}
6. MetaActual = c
7. Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {}
8. Verificar(MetaActual, BH) // Se verifica, c ya en la BH
9. Añadir(d, BH)

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:       Verificado = Verdadero;
11:       mientras NoVacío(NuevasMetas) y Verificado hacer
12:          MetaActual = SeleccionarMeta(NuevasMetas);
13:          Eliminar(MetaActual, NuevasMetas);
14:          Verificado = Verificar(MetaActual, BH);
15:       fin mientras
16:       si Verificado entonces
17:          Añadir(Meta, BH);
18:       fin si
19:    fin mientras
20:    devolver Verificado;
21: fin si
```

Meta a alcanzar: *d*

BH: {b, c, d}



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 4

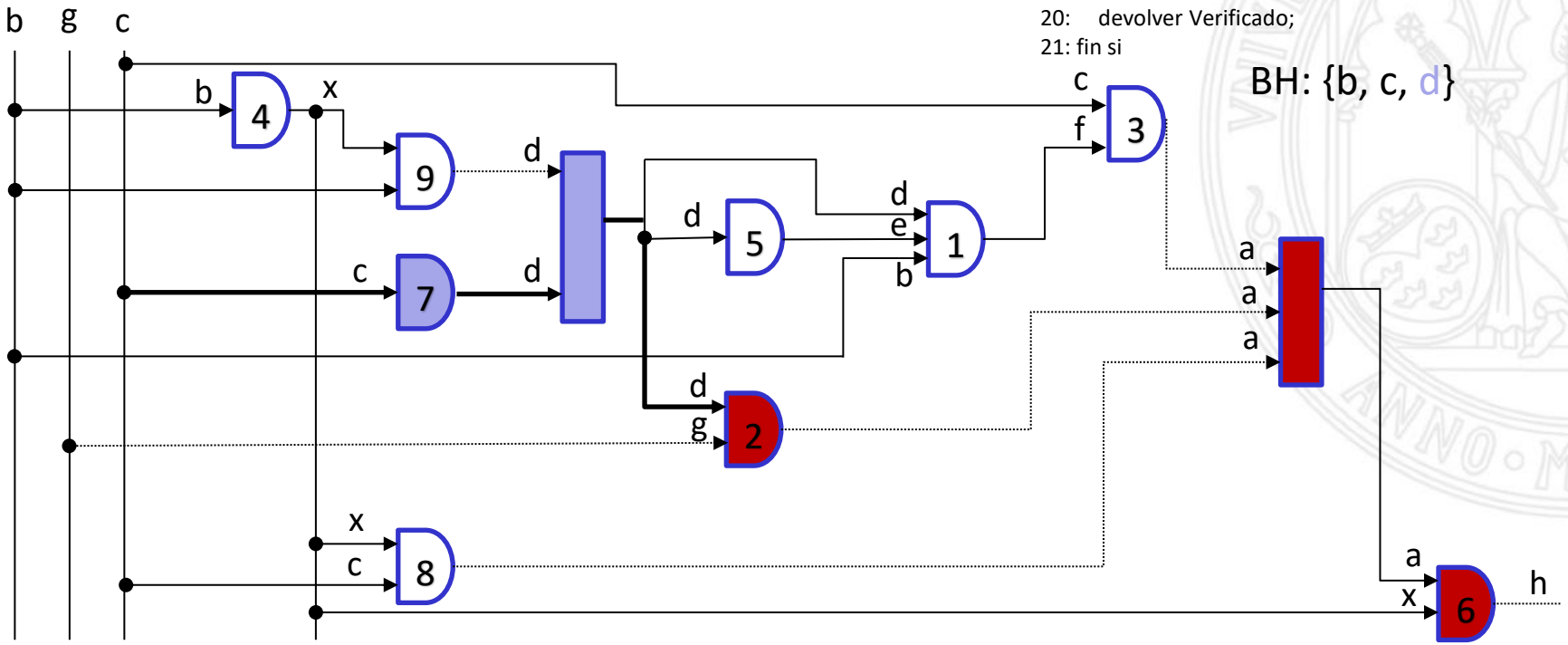
UNIVERSIDAD DE MURCIA

1. ConjuntoConflicto= {} // No está en la BH, no se puede verificar

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:       Verificado = Verdadero;
11:       mientras NoVacío(NuevasMetas) y Verificado hacer
12:          MetaActual = SeleccionarMeta(NuevasMetas);
13:          Eliminar(MetaActual, NuevasMetas);
14:          Verificado = Verificar(MetaActual, BH);
15:       fin mientras
16:       si Verificado entonces
17:          Añadir(Meta, BH);
18:       fin si
19:    fin mientras
20:    devolver Verificado;
21: fin si
```

Meta a alcanzar: *g*

BH: {b, c, d}



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 5

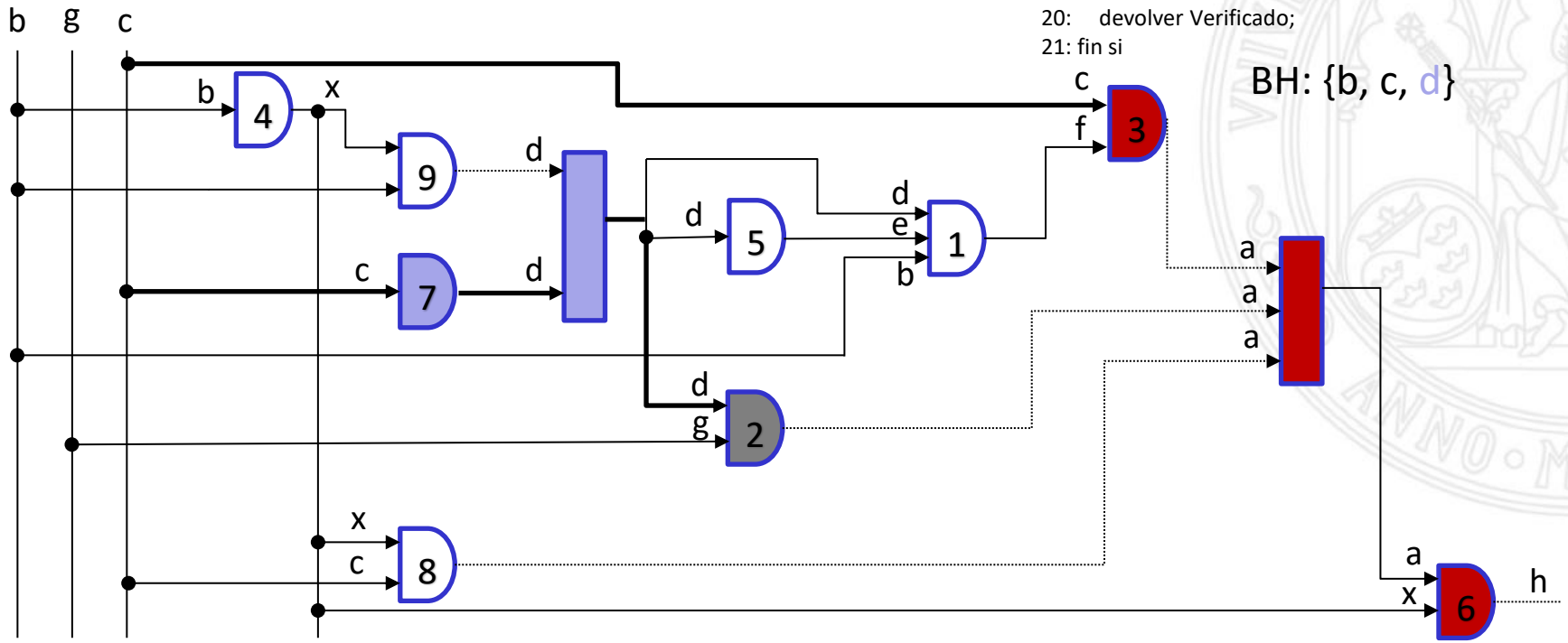
UNIVERSIDAD DE MURCIA

1. Meta = a
2. ConjuntoConflicto= {3, 8}
3. R = 3 // Resolución conflicto por orden “c & f → a”
4. Eliminar (R, CC) → ConjuntoConflicto= {8}
5. NuevasMetas= {c, f}
6. MetaActual = c
7. Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {f}
8. Verificar(MetaActual, BH) // Se verifica, c ya en la BH

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:      Verificado = Verdadero;
11:      mientras NoVacío(NuevasMetas) y Verificado hacer
12:         MetaActual = SeleccionarMeta(NuevasMetas);
13:         Eliminar(MetaActual, NuevasMetas);
14:         Verificado = Verificar(MetaActual, BH);
15:      fin mientras
16:      si Verificado entonces
17:         Añadir(Meta, BH);
18:      fin si
19:   fin mientras
20:   devolver Verificado;
21: fin si
```

Meta a alcanzar: *a*

BH: {b, c, d}



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 5

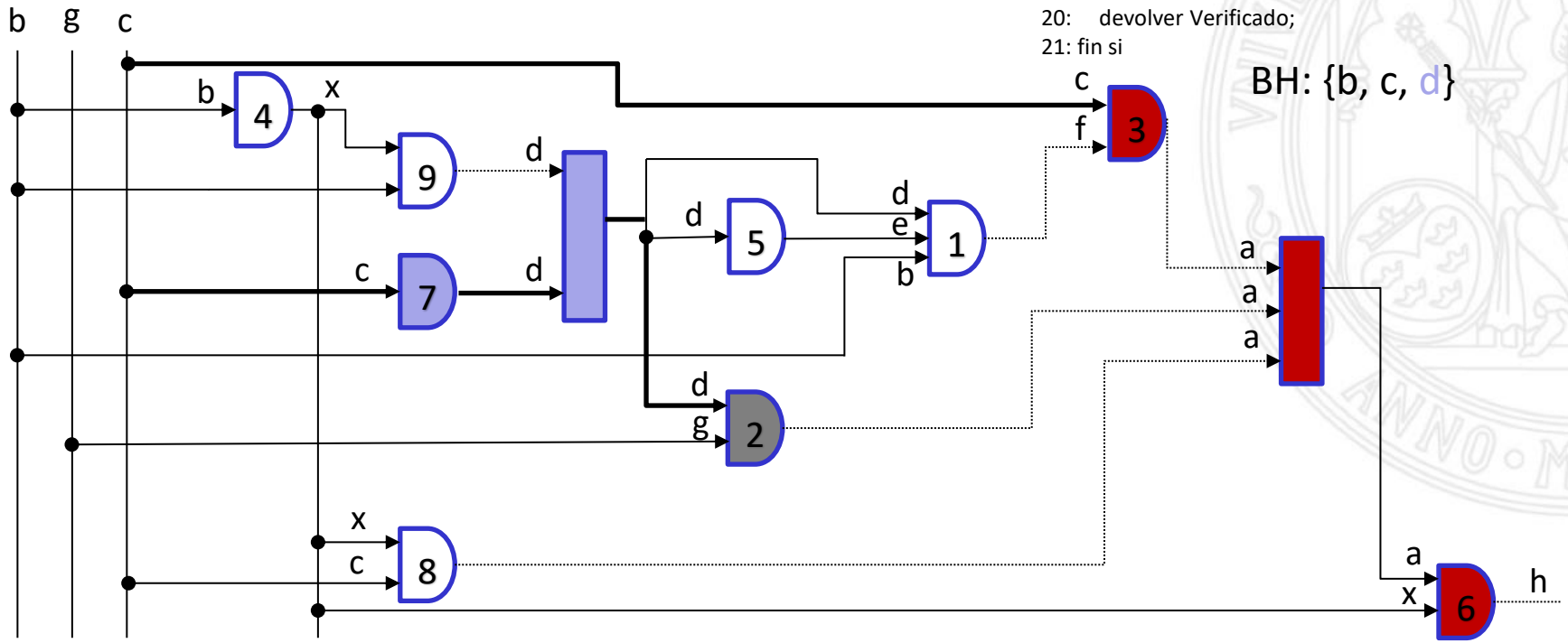
UNIVERSIDAD DE MURCIA

1. Meta = a
…
4. NuevasMetas= {f}
5. MetaActual = f
6. Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {}
7. Verificar(MetaActual, BH)

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:      Verificado = Verdadero;
11:      mientras NoVacío(NuevasMetas) y Verificado hacer
12:         MetaActual = SeleccionarMeta(NuevasMetas);
13:         Eliminar(MetaActual, NuevasMetas);
14:         Verificado = Verificar(MetaActual, BH);
15:      fin mientras
16:      si Verificado entonces
17:         Añadir(Meta, BH);
18:      fin si
19:   fin mientras
20:   devolver Verificado;
21: fin si
```

Meta a alcanzar: *a*

BH: {b, c, d}



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Ciclo 6

UNIVERSIDAD DE MURCIA

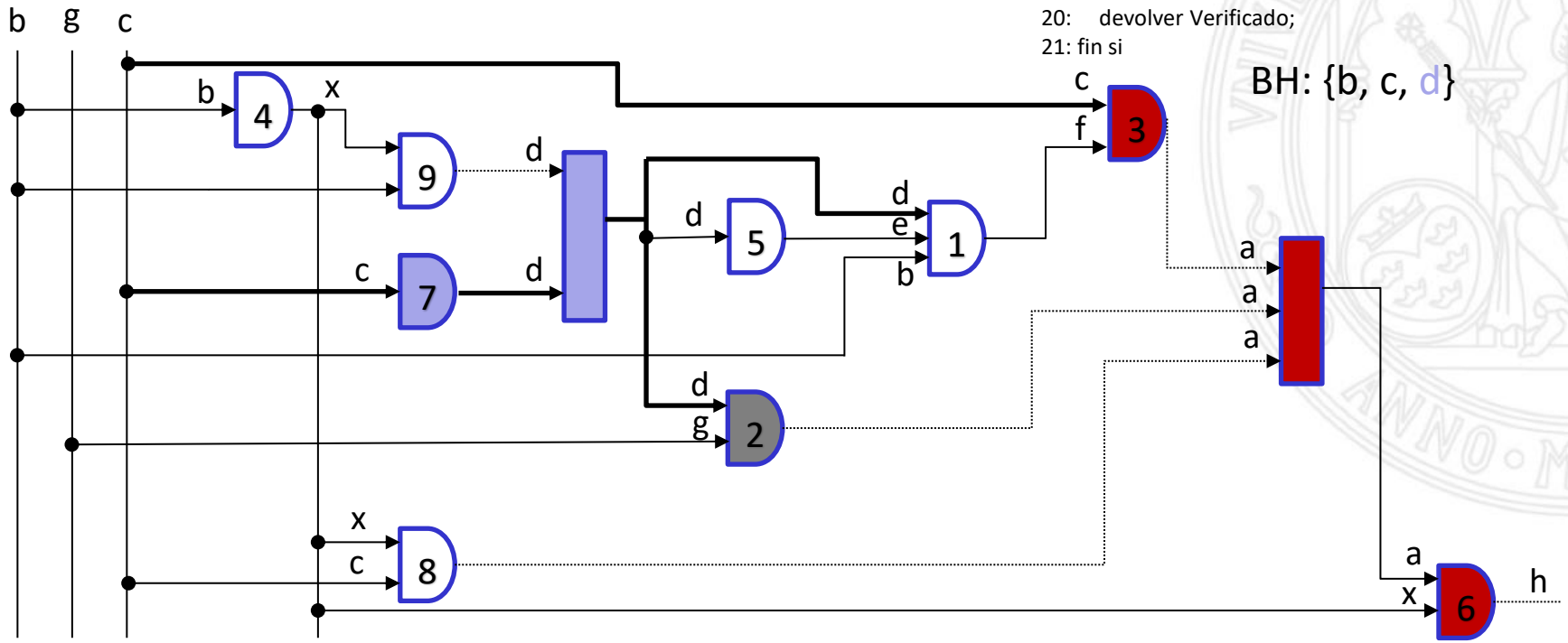
1. Meta = f
2. ConjuntoConflicto= {1}
3. R = 1 // Resolución conflicto por orden “b & d & e → f”
4. Eliminar (R, CC) → ConjuntoConflicto= {}
5. NuevasMetas= {d, e, b}
6. MetaActual = d
7. Eliminar(MetaActual, NuevasMetas) → NuevasMetas= {e, b}
8. Verificar(MetaActual, BH) // Se verifica, d ya en la BH

…

```
1: Verificado = Falso;
2: si Contenida(Meta, BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC), Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R = Resolver(ConjuntoConflicto);
8:       Eliminar(R, ConjuntoConflicto);
9:       NuevasMetas = ExtraerAntecedentes(R);
10:      Verificado = Verdadero;
11:      mientras NoVacío(NuevasMetas) y Verificado hacer
12:         MetaActual = SeleccionarMeta(NuevasMetas);
13:         Eliminar(MetaActual, NuevasMetas);
14:         Verificado = Verificar(MetaActual, BH);
15:      fin mientras
16:      si Verificado entonces
17:         Añadir(Meta, BH);
18:      fin si
19:   fin mientras
20:   devolver Verificado;
21: fin si
```

Meta a alcanzar: f

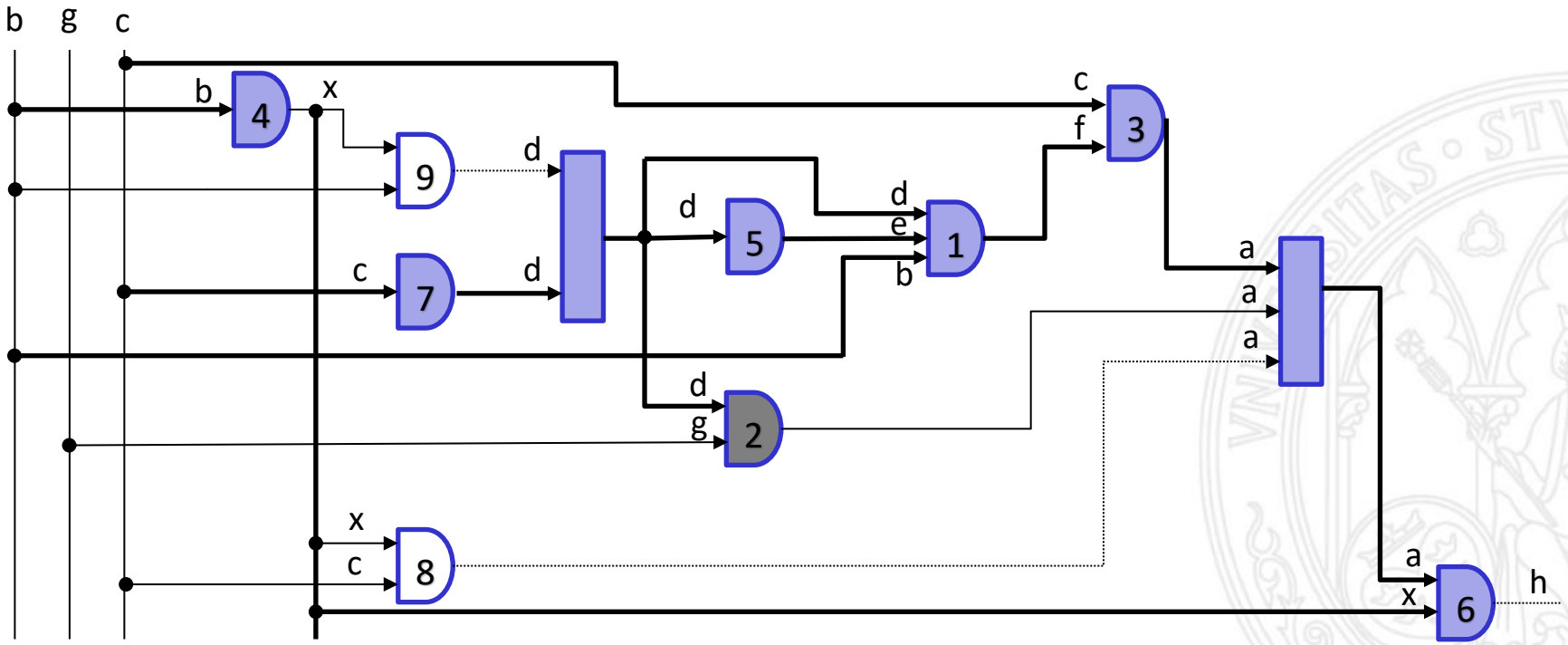
BH: {b, c, d}



Unidad 2. Reglas en la minería de datos
Ejemplo algoritmo hacia atrás: Estado final

UNIVERSIDAD DE MURCIA

BH: {b, c, d, f, a, x, h}



Unidad 2. Reglas en la minería de datos
Ejercicio 3: Completa el ejemplo

UNIVERSIDAD DE MURCIA

- Completa los ciclos del ejemplo anterior que se quedo incompleto.

Resumen encadenamiento hacia atrás

- Etapa de equiparación/matching: búsqueda de reglas cuya conclusión coincide con la meta M en curso
- Etapa de Resolución de conflictos: selección de una regla entra las dadas por el conjunto conflicto
- Etapa de acción: reemplazar la meta M por la conjunción de las condiciones del antecedente de la reglas seleccionada
- Problema: entrar en bucles infinitos, donde una meta, es a su vez sub-meta de su propio árbol de búsqueda
- Ventajas:
  - El sistema sólo plantea cuestiones al usuario cuando tiene necesidad
  - Limita el número de equiparaciones, ya que se centran en las necesarias para verificar y reduce la dimensión/tamaño del árbol de búsqueda dado

Unidad 2. Reglas en la minería de datos
Elección entre hacia delante o hacia atrás

- Depende cómo los expertos resuelven el problema
- Si primero obtiene la información y a partir de ahí intenta inferir qué puede estar pasando: adelante
  - Es la forma natural de diseñar sistemas para análisis e interpretación
- Si primero plantea una hipótesis (posible solución) y entonces intenta comprobar si es posible a partir de los datos disponible: hacia atrás
  - La mayoría se usa para propósitos de diagnóstico
- Muchos sistemas usan una combinación de ambas técnicas de inferencia

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

RESOLUCIÓN DE CONFLICTOS

- Definíamos resolución de conflictos como el método para elegir una regla cuando más de una podría ser lanzada
- La forma obvia de resolver un conflicto es establecer una meta y parar cuando se alcance
- Previamente hemos usado como criterio el orden de las reglas en la BC
  - Sin embargo, eso le da demasiada importancia al orden en el que se introdujeron
  - ¿Tiene sentido esto?
- Hay otras aproximaciones que son posibles

UNIVERSIDAD DE MURCIA

- Se puede lanzar la regla con prioridad más alta
- Esto podría hacerse situando las reglas en dicho orden
  - Esto funciona bien con pocas reglas, ¿pero que pasa si hay miles?
- Los sistemas de programación de reglas suelen permitir asignar prioridades

Goal 1. Prescription is? Prescription

RULE 1 Meningitis Prescription1
(Priority 100)
IF Infection is Meningitis
AND The Patient is a Child
THEN Prescription is Number_1
AND Drug Recommendation is Ampicillin
AND Drug Recommendation is Gentamicin
AND Display Meningitis Prescription1

RULE 2 Meningitis Prescription2
(Priority 90)
IF Infection is Meningitis
AND The Patient is an Adult
THEN Prescription is Number_2
AND Drug Recommendation is Penicillin
AND Display Meningitis Prescription2

Estrategia de equiparación más larga

- Se basa en la asunción de que una regla más específica procesa más información que una general
- Por lo tanto, priorizamos ejecutar aquellas que tengan un antecedente más largo

Rule 1:
IF the season is autumn
AND the sky is cloudy
AND the forecast is rain
THEN the advice is 'stay home'

Rule 2:
IF the season is autumn
THEN the advice is 'take an umbrella'

UNIVERSIDAD DE MURCIA

- Encuentra la regla que usa datos de la BH más actualizados
- Se basa en marcas de tiempo usadas para etiquetar los hechos
- Si hay un conflicto, se lanza aquella regla que el antecedente usa hechos más recientemente añadidos a la BH

Rule 1:
IF the forecast is rain [08:16 PM 11/25/96]
THEN the advice is 'take an umbrella'

Rule 2:
IF the weather is wet [10:18 AM 11/26/96]
THEN the advice is 'stay home'

Unidad 2. Reglas en la minería de datos
Meta-reglas

UNIVERSIDAD DE MURCIA

- Los criterios previos normalmente funcionan decentemente
- Cuando la complejidad crece, el ingeniero tiene mucho trabajo
- Hay que proporcionarle al sistema meta-conocimiento, es decir, conocimiento sobre el conocimiento
  - Esto se hace a través de meta-reglas, que determinen estrategia
- Lo hace el ingeniero conforme va conociendo el sistema experto y dominio, por ej:
  - Las reglas de expertos tienen mayor prioridad que novatos
  - Las reglas para rescatar vidas humanas tienen mayor prioridad que aquellas que limiten sobrecargas
- Algunos sistemas expertos proporcionan un motor de inferencia separado, pero normalmente no pueden distinguir
  - Se les debe dar más prioridad
  - Inyectan información en la BH al ser lanzadas para modificar reglas

UNIVERSIDAD DE MURCIA

Unidad 2. Reglas en la minería de datos

VENTAJAS Y DESVENTAJAS

Unidad 2. Reglas en la minería de datos
Comparativa

UNIVERSIDAD DE MURCIA

	Expertos humanos	Sistemas expertos	Programas convencionales
Procesado	Usan conocimiento en forma de “rule of thumb” o heurísticas para resolver problemas en un dominio concreto	Procesa el conocimiento expresado en forma de reglas y usa un razonamiento simbólico para resolver problemas en un dominio concreto	Procesa datos y usa algoritmos, mediante operaciones bien definidas, para resolver problemas generales numéricos
Conocimiento	En el cerebro humano, el conocimiento existe de forma compilada	Proporciona una clara separación entre el conocimiento y como se procesa	No separa conocimiento de las estructuras de control para procesarlo
Explicación	Capaz de explicar una línea argumental de razonamiento y proveer detalles	Traza las reglas lanzadas durante una sesión de resolución y explica una conclusión particular por la que se llegó a datos específicos	No explica como un resultado particular fue obtenido y porque los datos de entrada eran necesarios

Unidad 2. Reglas en la minería de datos
Comparativa

UNIVERSIDAD DE MURCIA

	Expertos humanos	Sistemas expertos	Programas convencionales
Ruido	Usa un razonamiento inexacto y puede gestionar información incompleta, incierta y borrosa	Permite el razonamiento inexacto y puede gestionar información incompleta, incierto y borroso	Sólo trabaja en problemas donde los datos son completos y exactos
Errores	Pueden cometer errores cuando la información es incompleta o borrosa	Pueden cometer errores cuando los datos son incompletos o borrosos	No proporcionan solución en absoluto, o una incorrecta, cuando los datos son incompletos o borrosos
Calidad	Mejora la calidad de las estrategias de resolución de problemas mediante años de aprendizaje y práctica. El proceso es lento, ineficiente y caro	Mejora la calidad de resolución de problemas añadiendo nuevas reglas o ajustando las antiguas. Cuando se obtiene nuevo conocimiento, los cambios son sencillos de implementar	Mejora la calidad de la resolución de problemas cambiando el código del programa, lo cual afecta a la base de conocimiento y su procesado, haciendo los cambios complicados

Unidad 2. Reglas en la minería de datos
Principales características

- Las siguiente son las principales características de los SBR para ingenieros del conocimiento
- Representación natural del conocimiento
  - Es sencillo pasar la explicación de un experto a un conjunto de reglas de producción
- Estructura uniforme
  - Tiene una estructura uniforme SI-ENTONCES
  - Cada regla es una pieza independiente de conocimiento
  - Su propia sintaxis es auto-documentada
- Separa el conocimiento de su procesado
  - Permite desarrollar distintas aplicaciones con el mismo sistema experto
  - Permite expandir el sistema fácilmente
  - Para hacer un SBR más inteligente, sólo añadimos más reglas sin intervenir en el motor de inferencia

Unidad 2. Reglas en la minería de datos
Principales problemas

UNIVERSIDAD DE MURCIA

- Relaciones opacas entre las reglas
  - Aunque las reglas son relativamente sencillas, su relación e interacción es opaca
  - Es difícil ver como una regla afecta la estrategia global, ya que no hay una estructura jerárquica
- Búsqueda ineficiente
  - El motor de inferencia hace una búsqueda exhaustiva en cada ciclo
  - Por lo que la búsqueda es lenta y pueden no ser buenos para aplicaciones de tiempo real
- Incapacidad para aprender
  - Los SBR no aprenden en base a la experiencia, a diferencia de un experto humano que sabe cuando romper una regla o modificarla
  - El ingeniero de conocimiento todavía es responsable de revisar y mantener el sistema